CHAPTER

1

Cloud Foundations

Service Models, Deployment Models, Virtualization, Global Infrastructure, Migration Economics and Operational Responsibility

Cloud Infrastructure and Services (25CS2206E) | Course Outcome 1 (CO1)

IN THIS CHAPTER

1.1	Cloud Service Models
1.2	Cloud Deployment Models
1.3	Virtualization
1.4	Global Cloud Infrastructure
1.5	The AWS Well-Architected Framework
1.6	Migration Strategies: The 7 Rs
1.7	The Cloud Migration Process
1.8	Total Cost of Ownership (TCO)
1.9	Tracing One HTTP Request Through a Multi-Tier System
1.10	Tri-Cloud Terminology Equivalence
1.11	Break-Fix Case Study: Responsibility Misattribution
	Chapter Summary, Key Terms, Review Questions, Lab Exercises and Further Reading

LEARNING OBJECTIVES

After studying this chapter, you will be able to:

- Explain the NIST definition of cloud computing and its five essential characteristics.
- Compare IaaS, PaaS, SaaS, FaaS and CaaS by control, responsibility and typical use.
- Distinguish public, private, community, hybrid and multi-cloud deployment models and select one for a given scenario.
- Describe how hypervisors, virtual machines and containers abstract physical hardware.
- Map the AWS global infrastructure (Regions, Availability Zones, edge locations) to availability and latency goals.
- Apply the six pillars of the AWS Well-Architected Framework to review a workload.
- Choose a migration strategy from the 7 Rs and build a total cost of ownership (TCO) and breakeven estimate.
- Trace a single HTTP request through a multi-tier system using distributed tracing concepts.
- Translate core terms between AWS, Microsoft Azure and Google Cloud.
- Diagnose break-fix incidents caused by misreading the shared responsibility model.

Chapter Introduction

A decade ago, a company that wanted to launch a new application first had to buy servers, find rack space, install operating systems and wait weeks before a single line of business code could run. Cloud computing removed most of that waiting. Compute, storage, networking and complete software products are now rented over the internet, provisioned in minutes and billed for what is actually used.

This chapter builds the vocabulary and mental models that the rest of the course depends on. We start with **what** is delivered (service models) and **where** it runs (deployment models). We then open the box and look at **how** one physical server is shared safely among many customers (virtualization), and how providers spread that capacity across the planet (global infrastructure). With the foundations in place, we turn to designing good systems (the Well-Architected Framework), moving existing systems to the cloud (the 7 Rs and the migration process) and deciding whether the move makes financial sense (total cost of ownership). The chapter closes with two operational topics that working engineers meet every week: following a request through a distributed system, and knowing who is responsible when something breaks.

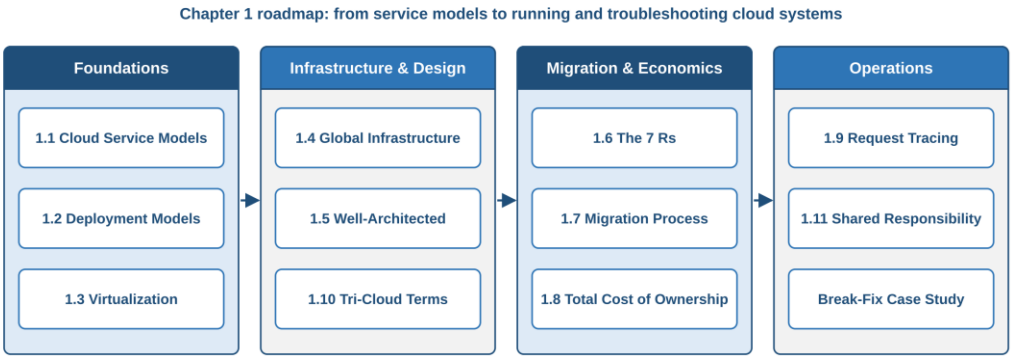


Figure 1.1 Chapter roadmap. Each group of sections builds on the one before it.

What Is Cloud Computing?

The most widely cited definition comes from the U.S. National Institute of Standards and Technology (NIST) in Special Publication 800-145:

> **DEFINITION: CLOUD COMPUTING (NIST SP 800-145)**
> "Cloud computing is a model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction."

NIST adds that the model is composed of five essential characteristics, three service models and four deployment models. The five characteristics are the test we can apply to any offering to decide whether it is really "cloud" or simply hosting with a new label. They are summarised in Table 1.1.

Table 1.1 The five essential characteristics of cloud computing (NIST SP 800-145)

Characteristic	Meaning	Everyday example
On-demand self-service	A consumer provisions resources alone, without a human at the provider approving each request.	Launching a virtual machine from a web console in two minutes.
Broad network access	Capabilities are reachable over the network through standard mechanisms from many client types.	Opening the same email account from a phone, laptop and tablet.
Resource pooling	The provider's resources serve many consumers in a multi-tenant model; the consumer does not know the exact physical location.	Thousands of customers sharing one data centre's servers.
Rapid elasticity	Capacity scales out and in quickly, often automatically, and appears unlimited to the consumer.	An exam-results website adding servers on results day and removing them the next week.

Characteristic	Meaning	Everyday example
Measured service	Usage is metered, monitored and reported, which makes pay-per-use billing possible.	A monthly bill that lists compute hours, gigabytes stored and data transferred.

NIST names three service models (IaaS, PaaS, SaaS) and four deployment models (private, community, public, hybrid). Industry practice has since added newer service models such as Function as a Service and Container as a Service, and a fifth deployment pattern, multi-cloud. We cover all of them in the next two sections.

1.1 Cloud Service Models

A **cloud service model** describes *how much* of the technology stack the provider operates on the customer's behalf. Deployment models, covered in Section 1.2, describe *where* the resources live and who may use them. The two ideas are independent: a company can consume SaaS from a public cloud, or run its own PaaS inside a private cloud.

> **REAL-WORLD ANALOGY: GETTING A PIZZA**
>
> Think of four ways to have pizza for dinner. You can make everything at home: dough, sauce, oven, table (**on-premises**). You can buy a take-and-bake pizza and use your own oven and table; the shop supplied the hard part of the ingredients (**IaaS**). You can order delivery and eat at your own table; you only provide the table and drinks (**PaaS**). Or you can go to a restaurant where everything is done for you and you simply eat (**SaaS**). In each step you give up some control over the recipe in exchange for less work.

Every computing service can be described as a stack of layers, from physical networking at the bottom to the application at the top. The service model decides which of those layers the customer manages and which the provider manages. Figure 1.2 shows this division, and it is the single most useful picture in this chapter: we will return to it when we discuss security responsibilities in Section 1.11.

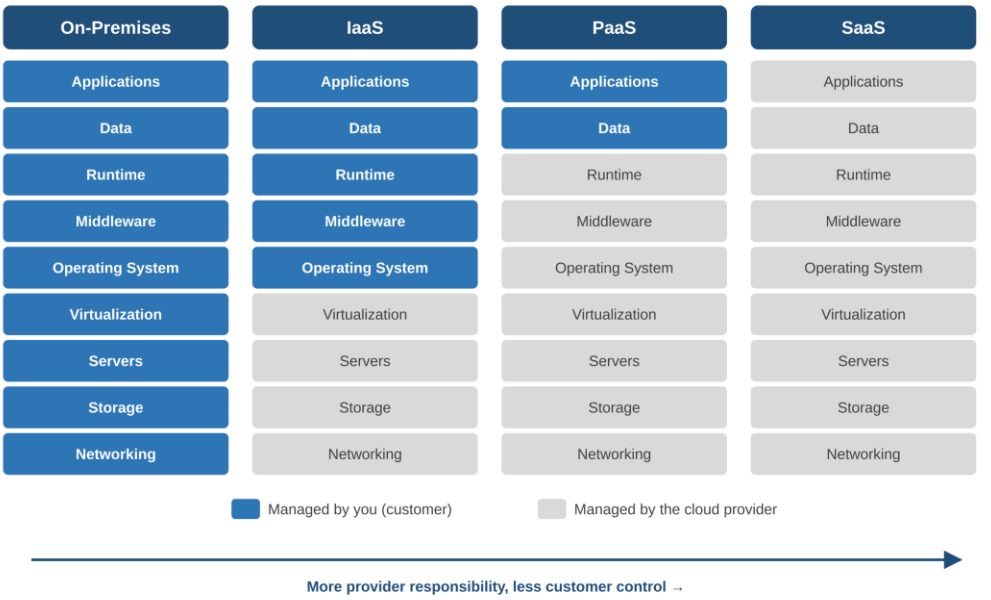


Figure 1.2 Division of management responsibility across the technology stack for on-premises, IaaS, PaaS and SaaS.

Reading the figure from left to right, the customer's share of the stack shrinks. With on-premises infrastructure the organization owns everything. With IaaS the provider takes

over the physical and virtualization layers. With PaaS it also takes over the operating system, middleware and runtime. With SaaS the customer manages only its own users, data and settings inside a finished application.

1.1.1 Software as a Service (SaaS)

> **DEFINITION: SAAS**
> **Software as a Service (SaaS)** delivers a complete application over the internet on a subscription basis. Users reach it through a web browser or a thin client, and the provider runs, updates and secures every layer of the stack beneath the user's data and configuration.

Because nothing is installed locally, a new user can be productive within minutes of receiving a login. The provider manages:

- Infrastructure: data centres, servers, storage and networking.
- Platform software: operating systems, middleware and databases.
- The application itself, including new features, bug fixes and security patches.
- Availability, backups of the platform, scaling and performance.

The customer remains responsible for who has access, how the application is configured, and the business data entered into it. That last point is the source of many incidents, as we will see in Section 1.11.

Multi-Tenant and Single-Tenant Architecture

A strategic decision in any SaaS product is how customers (called **tenants**) share the underlying resources. The choice affects cost, scalability, compliance, customization and the pace of upgrades. Figure 1.3 contrasts the two designs.

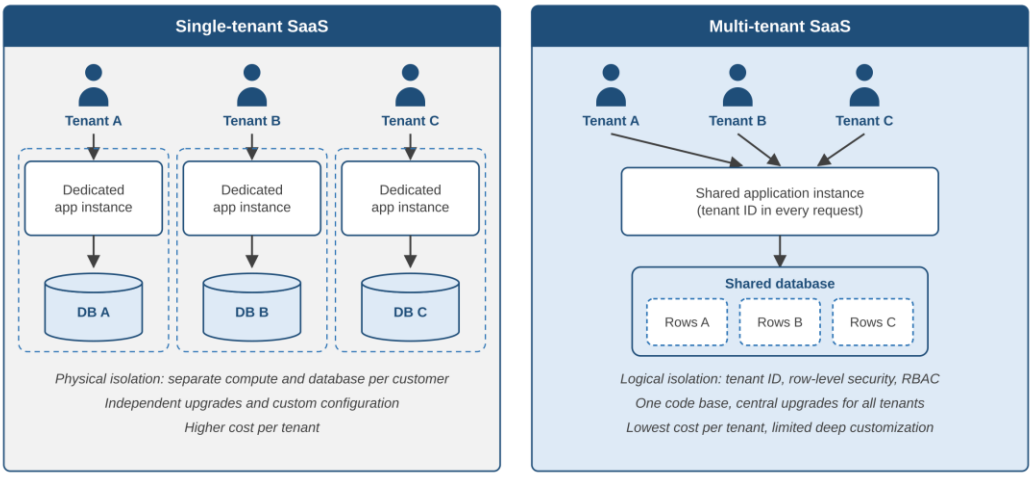


Figure 1.3 Single-tenant SaaS gives each customer dedicated compute and database resources; multi-tenant SaaS shares one application and database with logical separation.

In a **single-tenant** model, each customer has a dedicated application instance and database. No infrastructure is shared, so data isolation is physical. Customers can apply specific configurations, integrations or compliance-driven workflows without affecting anyone else, and upgrades can be scheduled per customer. This matters in regulated industries such as pharmaceuticals, where software changes often require formal validation before release. The cost is higher: every tenant needs its own monitoring, patching and support, so the provider's operating effort grows with each new customer.

In a **multi-tenant** model, many customers share the same application instance and infrastructure while their data is separated logically. Isolation is enforced through tenant identifiers carried in every request, row-level security in the database, role-based access control and careful application design. Updates are deployed once for everyone, which lowers cost and speeds innovation. The trade-offs are limited deep customization and a heavy responsibility on the provider to prevent any leakage of data between tenants.

> **REAL-WORLD ANALOGY: HOUSES AND APARTMENTS**
>
> Single tenancy is a row of detached houses: each family has its own walls, plumbing and front door, and can renovate freely. Multi-tenancy is an apartment block: families share the structure, lifts and water supply, which makes each home cheaper, but nobody can move a load-bearing wall.

Table 1.2 Multi-tenant versus single-tenant SaaS

Feature	Multi-tenant	Single-tenant
Infrastructure	Shared by all customers	Dedicated to one customer
Cost per customer	Lower	Higher
Customization	Configuration only	Extensive, including code-level changes
Data isolation	Logical (tenant ID, row-level security)	Physical (separate instance and database)
Upgrade control	Vendor schedules one upgrade for all	Customer can schedule its own upgrade
Noisy-neighbour risk	Possible; mitigated by quotas	None from other tenants
Compliance fit	Good for most sectors	Stronger for heavily regulated sectors

Examples of SaaS

Table 1.3 Representative SaaS products by category

Category	Products	What users do with them
Customer relationship management	Salesforce, HubSpot	Track leads, sales opportunities and customer service cases.
Collaboration and productivity	Google Workspace, Microsoft 365, Slack	Email, documents, spreadsheets, meetings and team chat.

Category	Products	What users do with them
Project management	Asana, Trello, Jira	Plan tasks, track progress and manage workflows.
Accounting and finance	QuickBooks Online, Xero, Zoho Books	Invoicing, bookkeeping and financial reporting.
Design and creative tools	Canva, Adobe Creative Cloud, Figma	Create graphics, edit images and design interfaces.
E-commerce	Shopify, BigCommerce	Build and run an online store without hosting it.
Enterprise and IT service management	ServiceNow, Workday	IT ticketing, HR, payroll and finance processes.

All of these products share the same defining traits: they are hosted by the vendor, reached through a browser or app, sold on a subscription, require no on-premises installation, and are maintained centrally.

Characteristics, Advantages and Limitations

Table 1.4 SaaS at a glance

Characteristics	Advantages	Limitations
▪ Usually multi-tenant ▪ Automatic updates and patching ▪ Subscription (per user or per usage) pricing ▪ Browser or mobile access ▪ Minimal configuration by the user	▪ No infrastructure to manage ▪ Fast adoption, often same day ▪ Predictable operating cost ▪ Access from anywhere ▪ Vendor carries uptime and security of the platform	▪ Little control over features and release timing ▪ Data residency depends on the vendor ▪ Integration limited to vendor APIs ▪ Switching vendors can be costly (data export, retraining)

SaaS is the easiest service model to adopt because it requires no infrastructure management at all. It is the right choice when an organization needs a standard business capability, such as email or payroll, rather than a differentiating system of its own.

1.1.2 Platform as a Service (PaaS)

> **DEFINITION: PAAS**
>
> **Platform as a Service (PaaS)** provides a managed environment in which developers build, test, deploy and run applications without managing the servers, storage, networks or operating systems underneath.

Developers concentrate on code and data; the platform takes care of provisioning, patching, scaling and much of the monitoring. A typical PaaS offering provides:

- An application hosting environment with language runtimes (Java, Node.js, Python, .NET, Go).
- A managed operating system that the provider patches.
- Database management systems and other managed services such as caches and message queues.
- Middleware for integration, messaging and identity.
- Development tools, deployment pipelines and command-line utilities.
- Built-in security features, load balancing and automatic scaling.

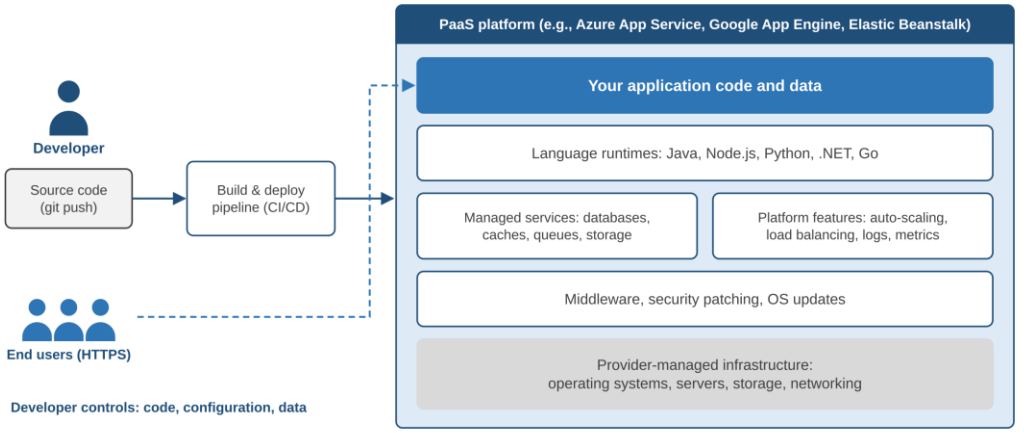


Figure 1.4 A PaaS platform. The developer pushes code through a pipeline; the provider operates everything below the application layer.

Examples of PaaS Platforms

- **Microsoft Azure App Service** for hosting web apps and APIs.
- **Google App Engine** for automatically scaled web applications.
- **AWS Elastic Beanstalk**, which deploys and scales applications on AWS resources from an uploaded package.
- **Heroku**, a developer-friendly platform known for deploying an application with a single `git push`.
- **Managed database platforms** such as Amazon RDS, Azure SQL Database and Cloud SQL, which apply the PaaS idea to databases.

Characteristics, Advantages and Limitations

Table 1.5 PaaS at a glance

Characteristics	Advantages	Limitations
▪ Built-in development tools ▪ Integrated CI/CD pipelines ▪ Automatic scaling ▪ Reduced infrastructure management ▪ Support for several programming languages	▪ Faster development and release cycles ▪ No OS patching or server sizing ▪ Built-in security and monitoring ▪ Lower operational overhead ▪ Good fit for agile and DevOps teams	▪ Less control over infrastructure and OS settings ▪ Risk of vendor lock-in through proprietary services ▪ Limited system-level customization ▪ Runtime versions follow the provider's support schedule

1.1.3 Infrastructure as a Service (IaaS)

> **DEFINITION: IAAS**
> **Infrastructure as a Service (IaaS)** provides virtualized computing resources (virtual machines, block and object storage, virtual networks, load balancers and firewalls) on demand over the internet. It offers the highest level of control of the three NIST service models.

The customer chooses the operating system image, installs middleware and applications, configures the network and is responsible for patching everything from the guest operating system upward. The provider guarantees the physical hosts, the hypervisor, the storage systems and the network fabric.

Major IaaS providers include **Amazon Web Services** (Amazon EC2, Amazon EBS, Amazon VPC), **Microsoft Azure** (Azure Virtual Machines, Managed Disks, Virtual Network) and **Google Cloud** (Compute Engine, Persistent Disk, VPC). IaaS closely resembles a traditional data centre, but without buying, racking or replacing hardware. Listing 1.1 shows how a single command creates a server.

Listing 1.1 Launching a virtual machine on AWS with the AWS CLI (IaaS)

```
# Launch one small Linux instance in the Hyderabad Region (ap-south-2)
aws ec2 run-instances \
    --region ap-south-2 \
    --image-id ami-0123456789abcdef0 \
    --instance-type t3.micro \
    --key-name my-keypair \
    --subnet-id subnet-0abc1234def567890 \
    --security-group-ids sg-0123abcd4567ef890 \
    --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=web-01}]'

# The customer now owns OS patching, software installation and data backup
```

The image, subnet and security group identifiers in the listing are placeholders; real values come from the customer's own account. Once the instance is running, the provider's job is to keep the host healthy. Everything inside the instance is the customer's job.

Table 1.6 IaaS at a glance

Characteristics	Advantages	Limitations
▪ High scalability ▪ Pay-as-you-go pricing ▪ Full control over OS and applications ▪ Virtualization-based infrastructure ▪ Flexible network and storage configuration	▪ No capital expenditure on hardware ▪ Capacity in minutes instead of weeks ▪ Familiar model for existing IT teams ▪ Good for lift-and-shift migrations ▪ Any software that runs on the OS can be used	▪ Customer patches and secures the OS ▪ Requires system administration skills ▪ Costs grow if instances are left running ▪ Design for availability is the customer's job

1.1.4 Comparing SaaS, PaaS and IaaS

Table 1.7 Key differences between the three NIST service models

Feature	SaaS	PaaS	IaaS
What you get	Ready-to-use software	A development and runtime platform	Virtualized infrastructure
User control	Minimal	Moderate	High
Who manages the OS	Provider	Provider	Customer
Typical users	Business end users	Developers	System administrators, IT teams
Customization	Low (configuration)	Medium (code and settings)	High (full stack above hypervisor)
Billing unit	Per user or per transaction	Per app instance, request or resource	Per instance-hour, GB-month, GB transferred
Skills needed	Application administration	Software development	Systems and network engineering
Example	Microsoft 365	Azure App Service	Amazon EC2

A short way to remember the difference: **SaaS** is ready-to-use software, **PaaS** is a development environment, and **IaaS** is virtualized infrastructure.

Benefits and Challenges of Cloud Service Models

Table 1.8 Benefits and challenges common to all cloud service models

Benefits	Challenges
▪ **Cost efficiency:** capital expenditure on hardware is replaced by operating expenditure. ▪ **Scalability:** resources scale up or down in minutes. ▪ **Accessibility:** services are reachable globally over the internet. ▪ **Automatic updates:** the provider maintains the layers it owns. ▪ **Disaster recovery:** built-in redundancy and backup options.	▪ **Data security and privacy:** data is held by a third party. ▪ **Vendor lock-in:** proprietary services make switching expensive. ▪ **Downtime risk:** a provider outage affects every customer of that service. ▪ **Compliance:** data residency and audit requirements must be met. ▪ **Limited customization:** especially for SaaS.

Use Cases for Each Model

Table 1.9 Typical use cases by service model

Model	Typical use cases
SaaS	Email and calendars, CRM, HR and payroll, collaboration tools, accounting.
PaaS	Web application and API development, mobile back ends, DevOps pipelines, managed databases.
IaaS	Hosting websites, running enterprise applications such as ERP, big-data analytics clusters, test labs, disaster recovery sites.

1.1.5 Emerging Service Models: FaaS and CaaS

Beyond the three NIST models, two newer models address the needs of cloud-native applications. Both sit between IaaS and SaaS on the control spectrum.

Function as a Service (FaaS)

> **DEFINITION: FAAS**
>
> **Function as a Service (FaaS)**, often called **serverless computing**, runs small pieces of code (functions) in response to events. The provider allocates compute only while a function runs and bills by the number of requests and the execution time.

There are still servers, of course; the developer simply never sees or manages them. Figure 1.5 shows the event-driven pattern.

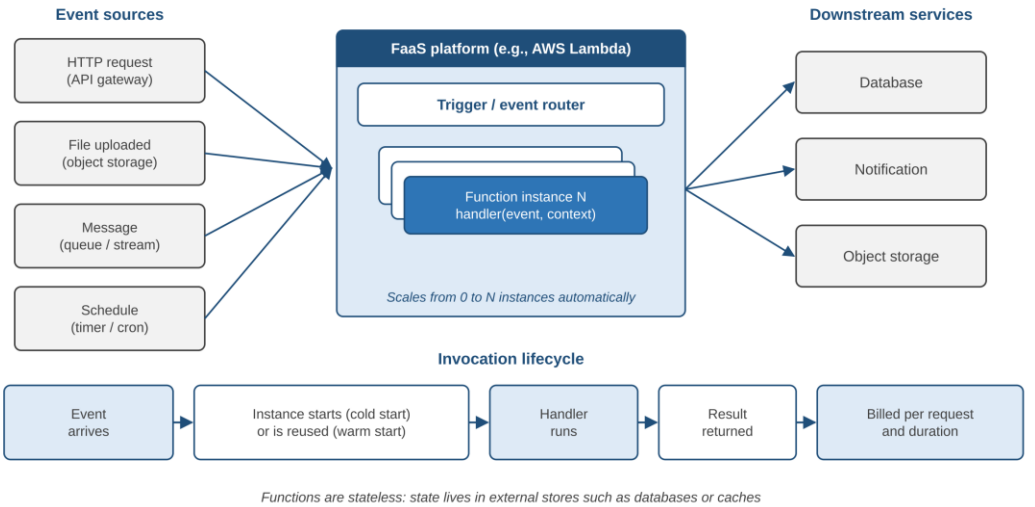


Figure 1.5 Function as a Service. Events trigger short-lived function instances, which scale automatically and call downstream services.

- **Event-driven execution:** functions start when an HTTP request, file upload, queue message or timer fires.
- **Automatic scaling:** from zero instances when idle to thousands during a spike.
- **Pay-per-execution pricing:** no charge when nothing runs.
- **Stateless functions:** any state that must survive between calls is stored in a database, cache or object store.
- **Cold starts:** the first request after a period of inactivity can take longer while a new execution environment is prepared.

Listing 1.2 A minimal AWS Lambda function in Python that responds to an API request

```
import json

def lambda_handler(event, context):
    # 'event' carries the HTTP request passed in by Amazon API Gateway
    name = (event.get("queryStringParameters") or {}).get("name", "student")
    return {
        "statusCode": 200,
        "headers": {"Content-Type": "application/json"},
        "body": json.dumps({"message": f"Hello, {name}!"})
    }
```

Popular FaaS platforms include AWS Lambda, Azure Functions and Google Cloud Run functions. FaaS suits microservices, file processing, scheduled jobs and other event-based systems with uneven traffic.

Container as a Service (CaaS)

> **DEFINITION: CAAS**
>
> **Container as a Service (CaaS)** lets organizations deploy, manage and scale containerized applications on a provider-managed container orchestration platform, usually Kubernetes, without operating the orchestration control plane themselves.

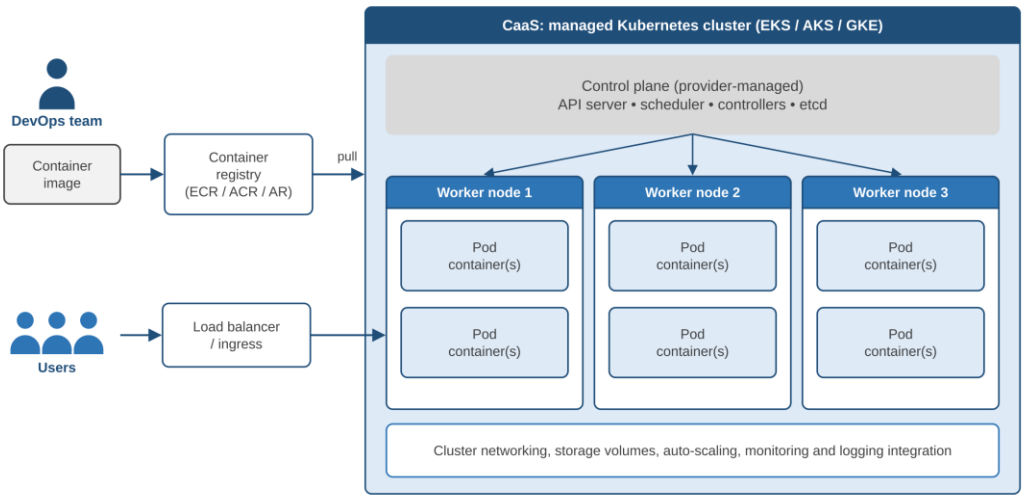


Figure 1.6 Container as a Service. The provider runs the Kubernetes control plane; the customer supplies container images and manages workloads on worker nodes.

CaaS platforms typically offer:

- A container runtime that runs images built with Docker or other OCI-compatible tools.
- Container orchestration, almost always Kubernetes.
- Cluster management, including upgrades of the control plane.
- Automatic scaling of pods and worker nodes.
- Load balancing and ingress for incoming traffic.
- Networking and persistent storage integration.
- Monitoring and logging tools.

Examples are Amazon Elastic Kubernetes Service (EKS), Azure Kubernetes Service (AKS) and Google Kubernetes Engine (GKE). Because containers package an application with its dependencies, the same image runs on a laptop, in a test cluster and in production, which gives CaaS high portability. It sits between IaaS and PaaS: more control than PaaS over the runtime, less operational work than building clusters on raw virtual machines.

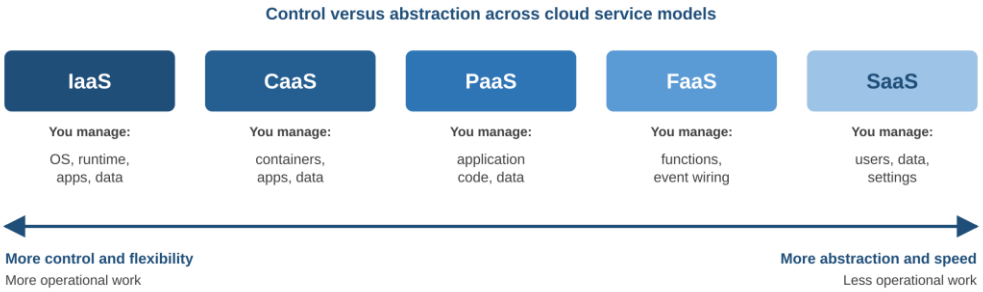


Figure 1.7 Control versus abstraction across the five service models.

Other “as a service” offerings, such as Database as a Service (DBaaS), Desktop as a Service (DaaS) and Backup as a Service, apply the same idea to a narrower capability. They are usually treated as specialized PaaS or SaaS products.

1.1.6 Choosing the Right Service Model

The right model depends on how much control the workload needs and how much operational effort the team can sustain. If regulation demands detailed infrastructure validation and audit trails at the operating-system level, IaaS may be the right fit. If the goal is rapid application development with moderate control, PaaS offers a balanced approach. If the aim is quick adoption of a proven business capability, SaaS is usually the best option. FaaS and CaaS suit teams building cloud-native, event-driven or containerized systems.

Table 1.10 A quick guide to selecting a service model

If the main requirement is...	Consider	Reason
A standard business function (email, CRM, payroll)	SaaS	No development or operations needed; fastest to adopt.
Building a web app or API quickly	PaaS	Provider handles OS, scaling and patching.
Short, event-driven tasks with uneven traffic	FaaS	Pay only per execution; scales to zero.
Portable microservices with fine runtime control	CaaS	Kubernetes portability without running the control plane.
Full control of OS, legacy software or special drivers	IaaS	Behaves like a server you own.

Six factors usually drive the decision: business objectives, budget, the team's technical expertise, security requirements, regulatory compliance and the degree of customization required. Most organizations end up using several models at once, for example SaaS for email, PaaS for customer-facing web applications and IaaS for a legacy ERP system.

Best Practices for Adopting Cloud Service Models

- Conduct a workload assessment before choosing a model for each application.
- Start with non-critical applications to build skills and confidence.

- Implement strong identity and access management from the first day.
- Define governance and compliance controls, including data classification.
- Reduce lock-in by preferring open standards and portable formats (containers, SQL, open APIs) where practical.
- Monitor performance and cost continuously, and tag every resource with an owner.

Future Trends

- **Edge cloud and distributed compute.** As IoT devices multiply and latency-sensitive workloads grow, providers extend cloud services closer to where data is produced and consumed. This supports real-time analytics, smart manufacturing and remote patient monitoring without sending every reading to a distant data centre.
- **Industry-specific clouds.** Providers now package services, data models and compliance controls for particular sectors, such as healthcare, financial services and manufacturing, so that customers start from an environment that already meets sector rules.
- **Cloud-native architecture.** Monolithic applications are being decomposed into microservices running in containers and orchestrated by Kubernetes, which improves resilience and release speed.
- **Serverless by default.** More application logic is written as functions and managed services, moving teams further toward code-centric development and away from server management.
- **AI-powered cloud services.** Providers are embedding machine learning and generative AI into their platforms, from managed model hosting to predictive auto-scaling, code assistants and intelligent security analytics.

Cloud service models have changed how organizations of every size consume technology. In practice, IaaS, PaaS, SaaS, FaaS and CaaS are combined into a single estate, and the skill that matters is choosing the right model for each workload.

1.2 Cloud Deployment Models

Deploying an application no longer requires an organization to own physical infrastructure. Cloud providers supply servers, storage and networks that can be combined into complete digital platforms. What differs between offerings is who owns the infrastructure, who may use it, where it is located and who controls its security. These differences are captured by the **cloud deployment model**.

> **DEFINITION: CLOUD DEPLOYMENT MODEL**
>
> A **cloud deployment model** defines where cloud infrastructure is located, who owns and operates it, and which users or organizations may access it. It determines how much the customer can change, how costs are structured and how security and compliance responsibilities are shared.

> **REAL-WORLD ANALOGY: PLACES TO STAY**
>
> A **public cloud** is a hotel: anyone can book a room, pay per night and leave. A **private cloud** is a house you own: nobody else enters, but you pay for the building and its upkeep. A **community cloud** is a gated housing society where several families share the security guards, garden and water tank. A **hybrid cloud** is owning a house and booking hotel rooms when relatives visit for a wedding. **Multi-cloud** is choosing rooms from several hotel chains so that one chain's problems never leave you without a bed.

NIST originally defined four deployment models: public, private, community and hybrid. Industry practice adds a fifth, multi-cloud. Figure 1.8 summarizes all five.

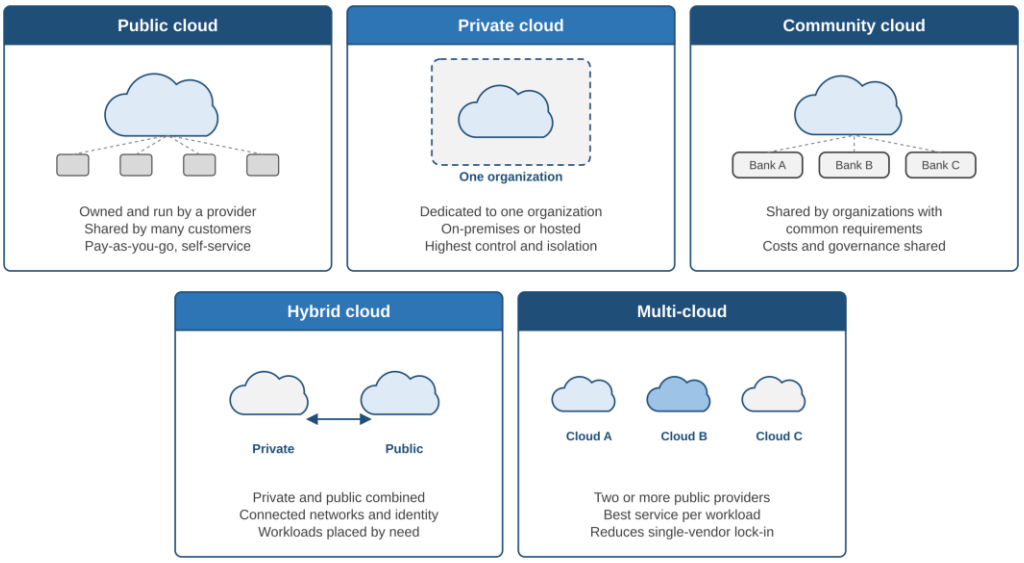


Figure 1.8 The five cloud deployment models.

1.2.1 Public Cloud

A **public cloud** is infrastructure owned and operated by a cloud service provider and made available to the general public over the internet. Customers share the provider's pooled hardware, but each customer's resources are isolated logically. Well-known examples are Amazon Web Services, Microsoft Azure, Google Cloud, Oracle Cloud Infrastructure and IBM Cloud. Note that “public” refers to who may sign up, not to who may see your data: resources in a public cloud are private to the account that owns them unless the owner chooses otherwise.

Because the provider owns the infrastructure, customers do not buy or maintain hardware. Resources are offered as services with pay-as-you-go pricing and can be scaled up or down as demand changes. The public cloud is a strong default for start-ups, small and medium businesses, and any workload with variable demand and no special isolation requirement.

Table 1.11 Advantages and disadvantages of the public cloud

Advantages	Disadvantages
▪ **Cost efficiency:** no hardware purchases; pay-as-you-go gives budget flexibility. ▪ **Simple scaling:** capacity follows demand without re-engineering. ▪ **Remote collaboration:** services are reachable from anywhere with an internet connection. ▪ **High availability:** many data centres with automated backups and failover options. ▪ **Provider-managed security of the platform:** patches and upgrades of provider-owned layers are handled for you. ▪ **Compliance support:** providers hold certifications such as ISO 27001, SOC 2 and PCI DSS for their infrastructure. ▪ **Easy integration:** large catalogues of ready-made services.	▪ **Limited control:** the customer cannot change the underlying infrastructure. ▪ **Security concerns:** sensitive data is held by a third party, so configuration mistakes are costly. ▪ **Service disruptions:** provider-side outages still happen, and customers can only wait or fail over. ▪ **Long-term expense:** steady 24 x 7 workloads may cost more over several years than owned hardware (see Section 1.8). ▪ **Hidden costs:** data egress, idle resources and premium support can accumulate. ▪ **Vendor lock-in:** proprietary services make migration to another provider harder. ▪ **Customization limits:** fixed instance types and service limits.

1.2.2 Private Cloud

A **private cloud** provides cloud capabilities (self-service, pooling, elasticity and metering) to a single organization. Technically it uses the same building blocks as a public cloud; the defining difference is that only one organization uses it. It may be hosted in the organization's own data centre or by a third party, and it is managed either by the organization's IT department or by a contracted provider. It is a common choice for businesses with strict data security, data sovereignty or legacy integration requirements. Platforms used to build private clouds include VMware Cloud Foundation, OpenStack and Azure Local (formerly Azure Stack HCI).

Table 1.12 Advantages and disadvantages of the private cloud

Advantages	Disadvantages
▪ **Fine-tuned security:** the owner controls every security protocol. ▪ **Data privacy:** only authorized users access data, stored on-premises or in a protected facility. ▪ **Customizability:** hardware, software and integrations can be shaped to the organization's needs. ▪ **Predictable performance:** no other tenants compete for resources; allocation can be tuned. ▪ **Long-term cost efficiency:** for stable, heavily used workloads, owned capacity can pay back its investment. ▪ **Legacy support:** integrates with systems that public clouds do not support.	▪ **Upfront investment:** hardware, facilities and licences must be bought before use. ▪ **Ongoing costs:** maintenance, support, upgrades and hardware refresh. ▪ **Labour-intensive scaling:** growth requires buying and installing more hardware. ▪ **Skilled staff:** specialists are needed to run and monitor the platform. ▪ **Slower to implement:** set-up takes weeks or months rather than minutes.

1.2.3 Community Cloud

A **community cloud** is shared by several organizations that have common requirements, such as security policies, compliance obligations or a joint mission. It is similar to a private cloud, except that ownership and use are spread across a defined group rather than one organization. Examples include clouds shared by banks under the same regulator, by government departments, by research universities or by hospitals that exchange patient records under the same health-data rules.

Table 1.13 Advantages and disadvantages of the community cloud

Advantages	Disadvantages
▪ **Shared cost:** infrastructure and support costs are divided among members. ▪ **Collaboration:** documents, data sets and dashboards are stored and accessed in one place. ▪ **Sector-specific design:** storage, security and audit controls are tailored to the community's regulations.	▪ **Slower scaling:** expansion needs agreement among members. ▪ **Complex governance:** upgrades, security policy and capacity allocation are decided jointly. ▪ **Cost:** usually cheaper than a private cloud but more expensive than a public cloud. ▪ **Dependency on agreements:** every major change needs consensus. ▪ **Privacy risk:** several organizations share the same infrastructure.

1.2.4 Hybrid Cloud

A **hybrid cloud** combines a private cloud (or on-premises infrastructure) with one or more public clouds. The environments remain distinct but are connected by secure networking,

shared identity and common management, so that data and applications can move between them. A typical design keeps sensitive data and core systems in the private environment while sending development and test workloads, analytics or sudden peaks in demand (**cloud bursting**) to the public cloud. Figure 1.9 shows the arrangement.

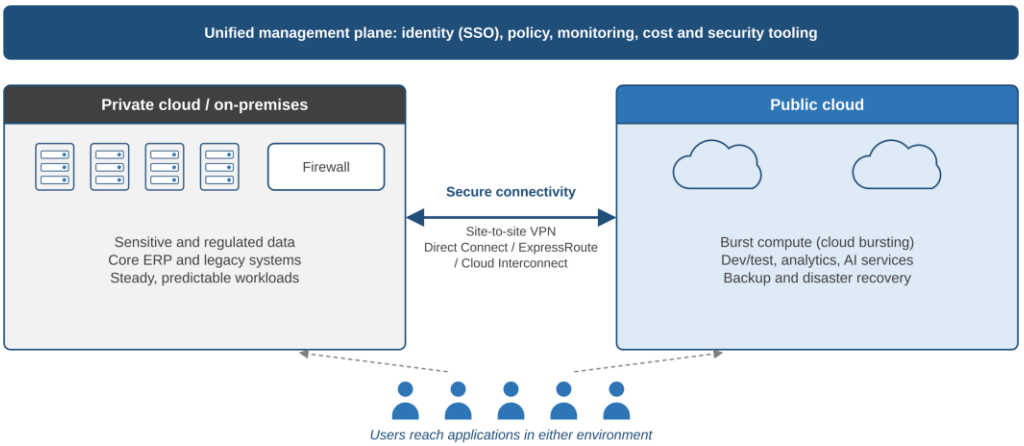


Figure 1.9 Hybrid cloud. Private and public environments are joined by secure connectivity and a common management plane.

Dedicated connectivity services make hybrid designs practical: AWS Direct Connect, Azure ExpressRoute and Google Cloud Interconnect provide private, high-bandwidth links between a customer's data centre and the provider. Hybrid platforms such as AWS Outposts, Azure Arc and Google Distributed Cloud extend provider services and management tools into the customer's own facilities.

Table 1.14 Advantages and disadvantages of the hybrid cloud

Advantages	Disadvantages
▪ **Flexible foundation:** sensitive data stays private while public resources add capacity. ▪ **Balanced cost:** only the sensitive or steady part of the estate needs owned infrastructure. ▪ **Easier disaster recovery:** the public cloud can host backups and standby environments. ▪ **Gradual migration:** workloads move to the public cloud at the organization's own pace.	▪ **Integration complexity:** networking, identity and monitoring must span both environments. ▪ **Higher set-up cost:** investment is needed on both sides plus the links between them. ▪ **Skills:** teams must understand private and public platforms. ▪ **Consistent security:** policies must be enforced the same way everywhere.

1.2.5 Multi-Cloud

A **multi-cloud** strategy uses services from two or more public cloud providers at the same time. It differs from hybrid cloud in that the environments involved are all public clouds (although an organization can be hybrid and multi-cloud at once). Organizations adopt multi-cloud to avoid dependence on one vendor, to use each provider's strongest services, to satisfy data residency rules in particular countries, or simply because different business units chose different providers. Figure 1.10 illustrates the pattern.

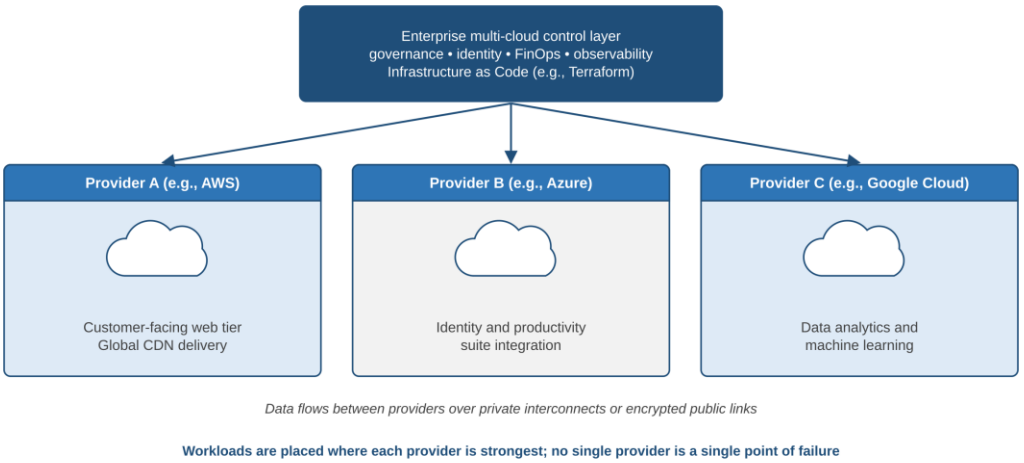


Figure 1.10 Multi-cloud. A common control layer governs workloads placed on several public providers.

Table 1.15 Advantages and disadvantages of multi-cloud

Advantages	Disadvantages
▪ **Avoids vendor lock-in:** the organization can move workloads if prices or terms change. ▪ **Optimized performance and cost:** each workload runs where it fits best. ▪ **Compliance and geographic reach:** data can be hosted in regions that meet local regulations. ▪ **Best-of-breed services:** for example, one provider for analytics and another for identity.	▪ **Management complexity:** monitoring, security and integration span several platforms. ▪ **Larger attack surface:** security policies must be adapted to each provider. ▪ **Compliance effort:** audits cover several environments with different controls. ▪ **Interoperability gaps:** services and APIs are not identical across providers. ▪ **Data transfer costs:** moving data between providers incurs egress charges.

KEY POINT: HYBRID IS NOT THE SAME AS MULTI-CLOUD

Hybrid means private plus public, connected and managed together. **Multi-cloud** means more than one public provider. An organization that runs an on-premises data centre, AWS and Azure is both hybrid and multi-cloud.

1.2.6 Comparing Deployment Models

Some organizations suit public clouds well; others avoid them because of project secrecy, intellectual-property protection or legal requirements. Table 1.16 compares the five models on the criteria most often used in selection.

Table 1.16 Comparison of cloud deployment models

Criterion	Public	Private	Community	Hybrid	Multi-cloud
Ownership	Provider	One organization	Member group	Mixed	Several providers
Upfront cost	None	High	Shared	Medium	None
Ease of set-up	Very easy	Hard	Moderate	Hard	Moderate
Scalability	Very high	Limited by owned hardware	Moderate	High (bursting)	Very high
Control over infrastructure	Low	Very high	High (shared)	High for private part	Low per provider
Isolation and data control	Logical	Physical	Shared within group	Choice per workload	Logical
Management complexity	Low	High	High (governance)	High	Highest
Typical users	Start-ups, SMEs, web apps	Banks, defence, large enterprises	Government, research, healthcare consortia	Enterprises in transition	Large enterprises, global firms

When choosing a deployment model, identify requirements first and then match them to the table:

- **Ease of configuration:** public, community and multi-cloud platforms are quickest because the provider does most of the work.
- **Data control:** strongest in private clouds, and in the private part of a hybrid cloud.
- **Reliability:** public clouds offer very high availability through multiple zones; private clouds reach it only with significant investment.
- **Scalability:** public, hybrid and multi-cloud designs scale furthest.
- **Security and confidentiality:** private and hybrid designs give the most direct control; public clouds can be equally secure when correctly configured.
- **Cost:** public and community clouds have the lowest entry cost because costs are shared.
- **Hardware requirements:** public and community clouds require none from the customer.

In general, the public cloud is the standard starting point for small and medium businesses. As organizations grow, regulation and steady workloads often make a private or hybrid component worthwhile, and large enterprises usually balance cost, control and risk with a combination of models.

1.3 Virtualization

Virtualization is the technology that makes cloud computing economically possible. Without it, every customer would need a dedicated physical server, and the pooling and elasticity described by NIST would not exist. In this section we look at what virtualization is, how it developed, how it is built, which tools manage it and how it is applied in practice.

> **DEFINITION: VIRTUALIZATION**
>
> **Virtualization** is the abstraction of physical computing resources (processors, memory, storage and networks) into software-defined virtual resources. It allows several isolated operating systems and applications to run on one physical machine, each believing it has the hardware to itself.

> **REAL-WORLD ANALOGY: AN APARTMENT BUILDING**
>
> A plot of land with one house wastes space if a single family uses two rooms. An apartment building divides the same plot into many self-contained flats, each with its own lock, kitchen and address. The building manager allocates water and electricity to each flat. The hypervisor is the building manager, the virtual machines are the flats, and the physical server is the plot of land.

Figure 1.11 compares a traditional server with a virtualized one. In the traditional model, one operating system owns the hardware, applications share libraries and can conflict, and most servers sit largely idle. In the virtualized model, a hypervisor divides the server into several virtual machines (VMs), each with its own guest operating system.

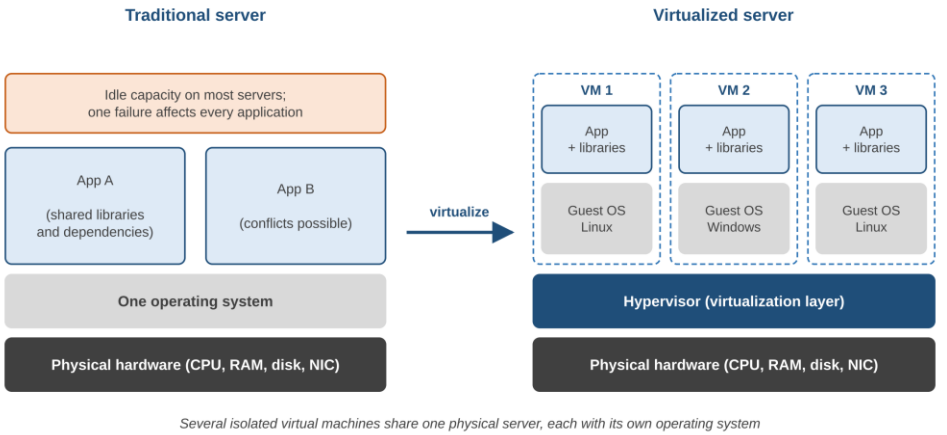


Figure 1.11 Traditional versus virtualized server architecture.

1.3.1 History of Virtualization

Virtualization is older than the personal computer. It began as a way to share expensive mainframes and became, half a century later, the foundation of the public cloud. Figure 1.12 and Table 1.17 trace the main milestones.

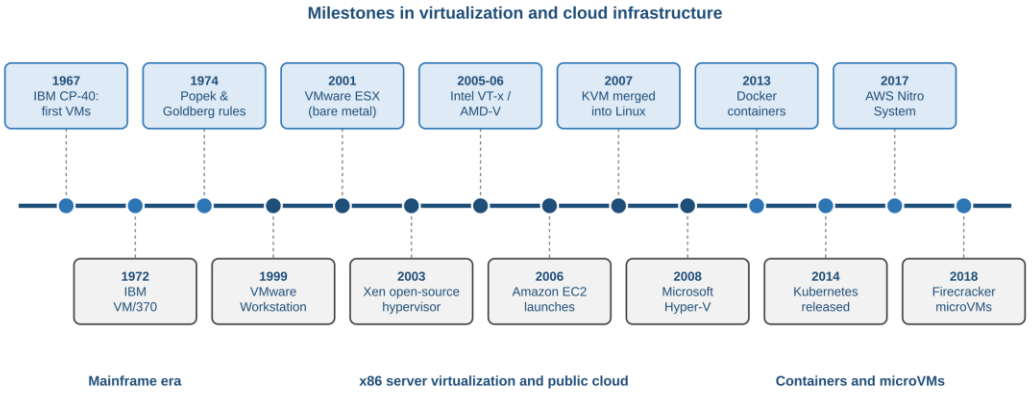


Figure 1.12 Milestones in the development of virtualization.

Table 1.17 Milestones in the history of virtualization

Period	Milestone	Significance
1960s	IBM CP-40 (1967) and CP-67	IBM's control programs let several users run isolated virtual machines on one mainframe, establishing the hypervisor concept.
1972	IBM VM/370	Brought full virtualization to commercial mainframe customers; VM descendants still run on IBM Z today.
1974	Popek and Goldberg requirements	Formal conditions for an architecture to be efficiently virtualizable (see Section 1.3.2).
1980s	Virtual memory in personal computers	Operating systems used disk space to extend RAM, popularizing software abstraction of hardware, although this is not machine virtualization.
1998-2001	VMware Workstation (1999) and ESX Server (2001)	Made x86 virtualization practical using binary translation, first on desktops and then on bare-metal servers.
2003	Xen	Open-source hypervisor using paravirtualization; later used by early Amazon EC2.
2005-2006	Intel VT-x and AMD-V	Hardware support for virtualization in x86 CPUs, removing the need for complex binary translation.
2006	Amazon EC2	Virtual machines sold by the hour; virtualization becomes the basis of public IaaS.
2007-2008	KVM in Linux; Microsoft Hyper-V	Hypervisors built into mainstream operating systems.
2013-2014	Docker and Kubernetes	Lightweight OS-level virtualization (containers) and their orchestration go mainstream.

Period	Milestone	Significance
2017-2018	AWS Nitro System; Firecracker microVMs	Virtualization functions offloaded to dedicated hardware; minimal VMs that start in a fraction of a second power serverless platforms.
2020s	AI-assisted operations, confidential computing	Machine learning used for placement and anomaly detection; hardware-encrypted VMs protect data even from the host.

Three turning points stand out. First, IBM proved in the 1960s and 1970s that one machine could safely host many isolated operating systems. Second, VMware showed around 2000 that the same idea could work on cheap x86 servers, which triggered large-scale server consolidation in data centres. Third, Amazon EC2 in 2006 turned virtualized capacity into a product sold by the hour, which is the moment cloud IaaS was born. Containers in the 2010s then offered a lighter form of isolation suited to microservices.

1.3.2 Core Concepts and Terminology

- **Hypervisor (virtual machine monitor):** software that creates and runs VMs. It sits between the hardware and the VMs, gives each VM the resources it needs and keeps VMs isolated from one another.
- **Virtual machine (VM):** a software emulation of a physical computer that runs its own operating system and applications.
- **Host and guest:** the host is the physical machine running the hypervisor; a guest is a VM running on that host, with its own guest OS.
- **Resource pooling:** CPU, memory, storage and network capacity from many hosts are pooled and allocated dynamically to VMs.
- **Isolation:** a fault, crash or compromise inside one VM should not affect other VMs on the same host.
- **Snapshot:** a saved point-in-time state of a VM's disk (and optionally memory) that can be restored later.
- **Live migration:** moving a running VM from one host to another with no noticeable downtime.
- **Overcommitment:** allocating more virtual CPUs or memory to VMs than the host physically has, relying on the fact that not all VMs peak at once.

When Can an Architecture Be Virtualized?

In 1974 Gerald Popek and Robert Goldberg stated three properties that a virtual machine monitor must have:

1. **Equivalence:** a program running under the monitor behaves essentially as it would on the real machine.
2. **Resource control:** the monitor is in complete control of the virtualized resources.

3. **Efficiency:** most machine instructions execute directly on the hardware without the monitor intervening.

They showed that these properties can be met when every *sensitive* instruction (one that reads or changes machine state) is also a *privileged* instruction that traps to the monitor. The original x86 architecture broke this rule: some sensitive instructions executed silently in user mode. This is why early x86 hypervisors needed software techniques such as binary translation, and why Intel VT-x and AMD-V were such an important step.

Table 1.18 Virtualization techniques

Technique	How it works	Example
Full virtualization (binary translation)	Unmodified guest OS; the hypervisor rewrites problematic instructions on the fly.	Early VMware Workstation and ESX
Paravirtualization	Guest OS is modified to call the hypervisor directly (hypercalls) instead of using privileged instructions.	Xen with paravirtualized Linux
Hardware-assisted virtualization	CPU extensions add a new privilege mode for the hypervisor, so unmodified guests run efficiently.	KVM, Hyper-V, modern ESXi on VT-x/AMD-V
OS-level virtualization	One kernel; isolated user spaces (namespaces) with resource limits (cgroups).	Docker, containerd, LXC

1.3.3 Types of Virtualization

Virtualization is applied to every major class of computing resource. Figure 1.13 groups the main types.

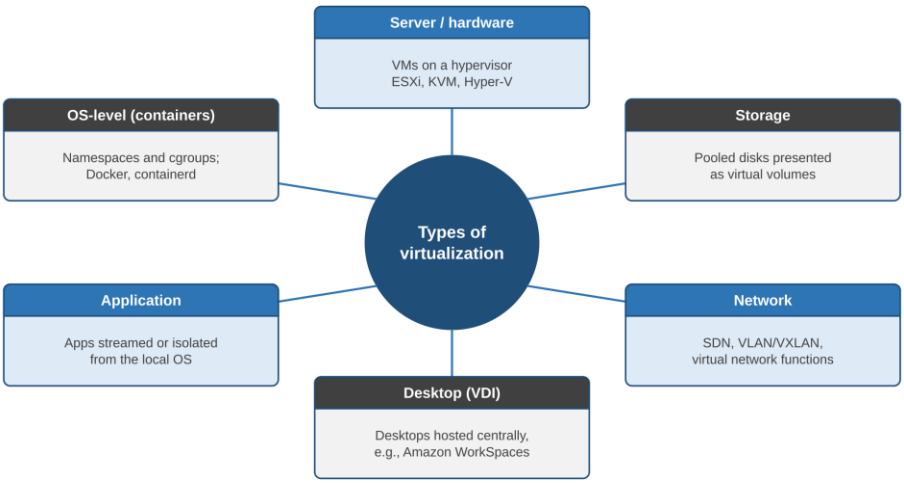


Figure 1.13 Types of virtualization.

Server (Hardware or Platform) Virtualization

The most fundamental form. A hypervisor creates virtual instances of a physical machine, so several operating systems run on one server. Each VM receives its own share of CPU, memory and devices. This type underpins data-centre consolidation and every IaaS compute service.

Storage Virtualization

Physical storage from many devices (hard drives, SSDs, SAN arrays or network-attached storage) is pooled and presented as logical volumes that can be allocated to VMs on demand. Administrators gain better utilization, simpler management and features such as thin provisioning and replication. Examples include VMware vSAN, Ceph and cloud block storage such as Amazon EBS.

Network Virtualization

Virtual networks are decoupled from the physical network. Bandwidth, IP addresses, switches, routers and firewalls are defined in software and provisioned independently of cabling. Network virtualization enables software-defined networking (SDN), overlay protocols such as VXLAN, and **network functions virtualization (NFV)**, where firewalls and load balancers run as software. Every cloud virtual private network (Amazon VPC, Azure Virtual Network, Google VPC) is an example.

Desktop and Application Virtualization

Desktop virtualization, or Virtual Desktop Infrastructure (VDI), runs user desktops on central servers and streams them to any device. It centralizes management and keeps data in the data centre. **Application virtualization** runs or packages applications so they are isolated from the local operating system, which avoids conflicts and simplifies maintenance. Examples include Citrix Virtual Apps and Desktops, Amazon WorkSpaces, Azure Virtual Desktop and Microsoft App-V.

OS-Level Virtualization (Containers)

Instead of emulating hardware, the operating system kernel creates isolated user spaces called containers. Linux **namespaces** give each container its own view of processes, network and file system, and **control groups (cgroups)** limit its CPU and memory. Containers are covered further in Section 1.3.7.

1.3.4 Virtualization Architecture

Type 1 and Type 2 Hypervisors

At the heart of virtualization architecture is the hypervisor. There are two main types, shown in Figure 1.14.

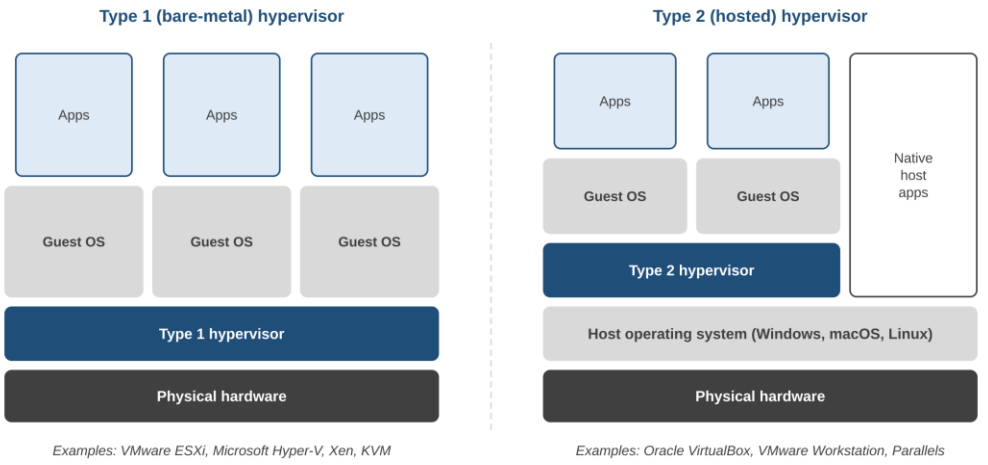


Figure 1.14 Type 1 (bare-metal) and Type 2 (hosted) hypervisors.

- **Type 1 (bare-metal) hypervisor:** runs directly on the host hardware without an underlying general-purpose operating system. It is efficient and secure and is the standard in enterprise data centres and public clouds. Examples are VMware ESXi (part of vSphere), Microsoft Hyper-V, Xen and KVM. KVM is a Linux kernel module that turns Linux itself into a Type 1 hypervisor.
- **Type 2 (hosted) hypervisor:** runs as an application on top of an existing operating system such as Windows, macOS or Linux. It is convenient for personal use, testing and training. Examples are Oracle VirtualBox, VMware Workstation and Parallels Desktop.

Table 1.19 Comparison of Type 1 and Type 2 hypervisors

Aspect	Type 1 (bare-metal)	Type 2 (hosted)
Runs on	Hardware directly	A host operating system
Performance	Near-native	Lower, because of the extra OS layer
Security and isolation	Smaller attack surface	Inherits host OS vulnerabilities
Typical use	Data centres, cloud providers	Developer laptops, labs, demonstrations
Examples	ESXi, Hyper-V, Xen, KVM	VirtualBox, VMware Workstation, Parallels

Virtual Machines and Resource Management

Each VM acts as a separate computer with its own OS and applications but shares the host's physical resources. Resource management is the hypervisor's central job: allocating CPU time, memory, storage I/O and network bandwidth so that every VM performs adequately and no VM starves the others (**resource contention**). Administrators use

reservations (guaranteed minimums), limits (maximums) and shares (relative priority) to control this.

CPU and Memory Virtualization

The hypervisor presents virtual CPUs, memory, disks and network interfaces to each VM. Hardware extensions such as **Intel VT-x** and **AMD-V** let guest code run directly on the processor while sensitive operations trap to the hypervisor. For memory, **second-level address translation** (Intel EPT, AMD NPT) lets the hardware translate guest addresses to host addresses without hypervisor involvement on every access. Techniques such as **memory ballooning** and page sharing reclaim unused memory from VMs and allow safe overcommitment.

1.3.5 Setting Up Virtual Environments

A reliable virtual environment depends on correct hardware, network and storage planning.

Table 1.20 Hardware and network requirements for a virtualization host

Component	Requirement
CPU	Multi-core processors with virtualization extensions (Intel VT-x or AMD-V) enabled in firmware.
RAM	Enough memory for all VMs plus hypervisor overhead; memory is usually the first resource to run out.
Network	Redundant, fast network interfaces (10 GbE or faster in data centres) for VM, storage and management traffic.
Storage	Fast, redundant storage (SSD or NVMe) for VM disks; shared storage for live migration.

Table 1.21 Storage planning considerations

Consideration	Description
Capacity	Enough space for all VM disks, snapshots and growth.
Redundancy	RAID, replication or erasure coding to survive disk failures.
Performance	SSDs or NVMe for I/O-intensive VMs; separate tiers for archive data.
Scalability	Ability to add capacity without downtime as the environment grows.

1.3.6 Virtualization Management Tools

Once an environment is built, management tools are needed to deploy, monitor, migrate and scale VMs. This subsection introduces the most widely used platforms with short command examples. The names of hosts, datastores and networks in the listings are examples and must be replaced with real values.

VMware vSphere

VMware vSphere is a widely deployed enterprise virtualization platform. Its main components are listed in Table 1.22. VMware is now part of Broadcom (since 2023), which changed product packaging and licensing; organizations planning new deployments should review current terms.

Table 1.22 Key components of VMware vSphere

Component	Purpose
ESXi	The Type 1 hypervisor installed on each physical host.
vCenter Server	Central management of many hosts, clusters and VMs.
vMotion	Live migration of running VMs between hosts with no downtime.
DRS and HA	Distributed Resource Scheduler balances load; High Availability restarts VMs after a host failure.

Listing 1.3 Creating, starting and live-migrating a VM with VMware PowerCLI

```
# Connect to vCenter
Connect-VIServer -Server vcenter01.lab.local

# Create a VM with 2 vCPUs, 4 GB RAM and a 40 GB disk
New-VM -Name "AppVM01" -VMHost "esxi01.lab.local" -Datastore "datastore1" `
       -NumCpu 2 -MemoryGB 4 -DiskGB 40 -NetworkName "VM Network"

# Power on the VM
Start-VM -VM "AppVM01"

# Live-migrate the running VM to another host (vMotion)
Move-VM -VM "AppVM01" -Destination (Get-VMHost "esxi02.lab.local")
```

Microsoft Hyper-V

Hyper-V is Microsoft's hypervisor, built into Windows Server and some Windows client editions. It integrates closely with Active Directory and System Center, supports live migration, and offers **Dynamic Memory**, which adjusts a VM's memory allocation according to demand.

Listing 1.4 Creating a Generation 2 VM in Hyper-V with PowerShell

```
# Create a VM with a new 40 GB virtual disk attached to a virtual switch
New-VM -Name "VM01" -MemoryStartupBytes 2GB -Generation 2 `
       -NewVHDPath "C:\VMs\VM01.vhdx" -NewVHDSizeBytes 40GB `
       -SwitchName "ExternalSwitch"

# Assign two virtual processors and enable Dynamic Memory
Set-VMProcessor -VMName "VM01" -Count 2
Set-VMMemory -VMName "VM01" -DynamicMemoryEnabled $true -MinimumBytes 1GB -MaximumBytes
4GB

Start-VM -Name "VM01"
```

KVM and QEMU

For Linux environments, **KVM** (Kernel-based Virtual Machine) provides hardware-assisted virtualization inside the Linux kernel, and **QEMU** provides device emulation and can emulate other CPU architectures. Together they form the basis of many private clouds (for example OpenStack) and of several public clouds. The `libvirt` toolkit and `virt-install` command simplify their management.

Listing 1.5 Creating a VM with KVM using virt-install, and running one directly with QEMU

```
# Install the tools (Debian/Ubuntu)
sudo apt-get install -y qemu-kvm libvirt-daemon-system virtinst

# Create a VM with 2 vCPUs, 2 GB RAM and a 10 GB disk from an installer ISO
virt-install --name MyVM --vcpus 2 --memory 2048 --disk size=10 \
    --cdrom /path/to/ubuntu-24.04.iso --os-variant ubuntu24.04

# Alternatively, boot a disk image directly with QEMU using KVM acceleration
qemu-system-x86_64 -enable-kvm -m 2048 -hda /path/to/disk.img \
    -cdrom /path/to/installer.iso -boot d
```

Docker

Docker packages an application and its dependencies into an image and runs it as a container that shares the host OS kernel. Instead of managing whole virtual machines, administrators manage containers, which start faster and use fewer resources.

Listing 1.6 Running a container with Docker

```
# Download the official Ubuntu image
docker pull ubuntu:24.04

# Start an interactive shell inside a new container
docker run -it --rm ubuntu:24.04 /bin/bash
```

Table 1.23 Comparison of virtualization tools

Tool	Platform	Type of virtualization	Best use case
VMware vSphere	Enterprise (runs Windows and Linux guests)	Full, hardware-assisted	Large enterprises needing mature management, HA and live migration
Microsoft Hyper-V	Windows Server	Full, hardware-assisted	Windows-centric estates integrated with Active Directory
KVM / QEMU	Linux	Full virtualization and emulation	Open-source private clouds (e.g., OpenStack), cost-sensitive deployments
Docker	Linux, Windows, macOS	OS-level (containers)	Microservices, DevOps pipelines, cloud-native applications

1.3.7 Virtual Machines and Containers

Containers and VMs both isolate workloads, but at different layers. Figure 1.15 and Table 1.24 compare them.

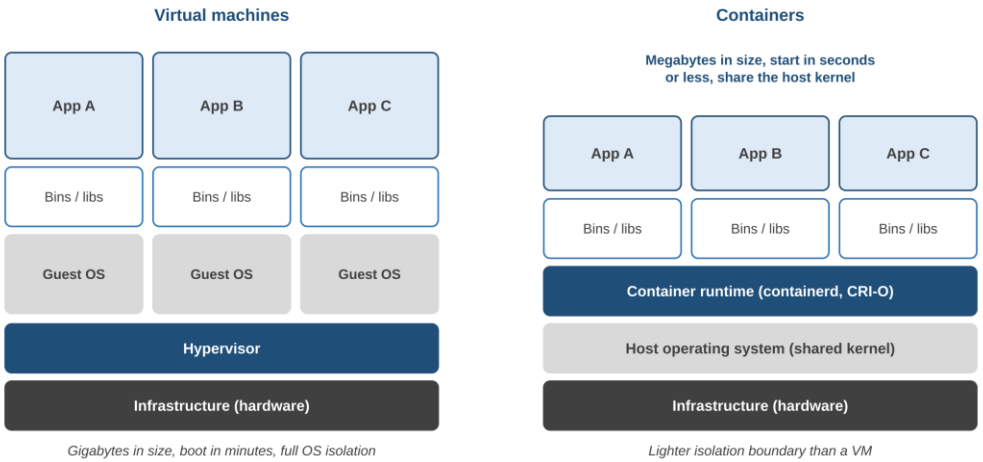


Figure 1.15 Virtual machines virtualize hardware; containers virtualize the operating system.

Table 1.24 Virtual machines versus containers

Aspect	Virtual machine	Container
Isolation boundary	Hardware (hypervisor)	Operating system kernel
Includes	Full guest OS, libraries, application	Libraries and application only
Typical size	Gigabytes	Megabytes
Start-up time	Tens of seconds to minutes	Milliseconds to seconds
Density per host	Tens	Hundreds
Security isolation	Stronger	Weaker by default; hardened with sandboxes or microVMs
Best for	Mixed operating systems, legacy apps, strong isolation	Microservices, CI/CD, portable cloud-native apps

Docker and Kubernetes

Docker made containers easy to build and share. Running hundreds or thousands of containers, however, needs automation. **Kubernetes** is an open-source container orchestration platform that schedules containers across a cluster, restarts failed ones, scales them with demand, balances traffic and rolls out new versions gradually. Managed Kubernetes services (EKS, AKS, GKE) are the CaaS offerings described in Section 1.1.5.

Microservices Architecture

Containers pair naturally with **microservices**, an architectural style in which an application is split into small, independent services that are developed, deployed and scaled

separately. Each service runs in its own container, so a team can update one service without redeploying the whole application, and a failure is contained to one service instead of bringing down the entire system.

1.3.8 Virtualization in Cloud, Edge and Industry

Cloud Integration

Virtualization is what turns a data centre into a cloud. It lets private, hybrid and multi-cloud environments (described in Section 1.2) share pooled capacity, move VMs between hosts and sites, and automate provisioning. Migration of virtualized workloads to the cloud is treated in detail in Section 1.6.

Virtualization for Edge Computing

Edge computing processes data close to where it is generated, which reduces latency and allows real-time decisions. Virtualization lets small edge servers run several isolated workloads that can be managed centrally.

- **IoT and smart factories:** containers on an edge gateway analyse sensor data locally and adjust machines in milliseconds, sending only summaries to the cloud.
- **Remote sites:** oil rigs, research stations and field hospitals run VMs or containers on compact hardware when links to a central data centre are slow or unreliable.
- **Mobile networks:** with 5G, operators run firewalls, load balancers and other network functions as virtual network functions (VNFs) at the network edge, and use network slicing to prioritize critical traffic.

Listing 1.7 Deploying an IoT monitor container and an OpenStack VNF instance on edge nodes

```
# Run an IoT monitoring application in a container on an edge gateway
docker run -d --name iot_sensor_monitor --restart unless-stopped -p 8080:80 iot-
monitor:latest

# Create a small virtual network function instance on an OpenStack edge site
openstack server create --flavor m1.small --image ubuntu-24.04 \
    --network edge-network --security-group default vnf-edge-node-01
```

Performance Optimization and Monitoring

Performance in a virtual environment depends on giving each VM what it needs without waste. Tools such as VMware DRS and Hyper-V resource metering balance load automatically, and monitoring tools such as VMware Aria Operations, Microsoft System Center and cloud-native services (Amazon CloudWatch, Azure Monitor) report consumption and reveal bottlenecks.

Table 1.25 Performance tuning guidelines

Resource	Guideline
CPU	Size vCPUs to the workload; too many vCPUs can increase scheduling delay (CPU ready time).

Resource	Guideline
Memory	Use dynamic memory or ballooning carefully; avoid host-level swapping.
Storage	Place I/O-heavy VMs on SSD or NVMe; spread disk I/O across datastores.
Network	Use paravirtualized drivers (e.g., VMXNET3, virtio) and separate traffic types.

Security in Virtual Environments

Sharing hardware creates new risks. **VM escape** is an attack in which code inside a VM breaks out to the hypervisor or host. **Cross-VM attacks** attempt to read data from neighbouring VMs, including through side channels. Mitigations combine strong isolation, segmentation, encryption of data at rest and in transit, regular patching of hypervisors and guests, and role-based access control (RBAC) for administrators.

Table 1.26 Security concerns and mitigations in virtual environments

Concern	Mitigation
VM escape	Patch hypervisors promptly; minimise hypervisor attack surface; use hardware-based isolation.
Cross-VM attacks	Network segmentation, micro-segmentation firewalls, dedicated hosts for sensitive workloads.
Unauthorized access	RBAC, multi-factor authentication for management consoles, regular permission audits.
VM sprawl	Inventory, tagging and automatic retirement of unused VMs and snapshots.

Regulated industries must also show compliance with standards such as HIPAA (healthcare), PCI DSS (payment cards) and data protection laws such as India's Digital Personal Data Protection Act, 2023. Encryption, monitoring and access controls in the virtual layer support those obligations.

Disaster Recovery and High Availability

Because a VM is a set of files, it can be copied, replicated and restarted elsewhere. Tools such as VMware Site Recovery Manager and Hyper-V Replica replicate VMs to another site and automate failover. **High availability (HA)** clusters restart VMs on surviving hosts when one host fails, and live migration lets administrators evacuate a host before maintenance.

Table 1.27 Disaster recovery and high availability

Strategy	Benefit
Disaster recovery (DR)	VM replication to another site reduces downtime and data loss after a major outage.
High availability (HA)	Automatic restart of VMs on healthy hosts keeps services running after hardware failure.

Next-Generation Virtualization

- **AI-assisted operations:** machine learning predicts demand, places VMs, detects anomalies and right-sizes resources before performance degrades.
- **MicroVMs:** minimal virtual machines such as AWS Firecracker start in a fraction of a second and give serverless functions VM-level isolation.
- **Hardware offload:** systems such as the AWS Nitro System move networking, storage and security functions to dedicated cards, leaving almost all host resources to customers.
- **Confidential computing:** technologies such as AMD SEV-SNP and Intel TDX encrypt VM memory so that even the host cannot read it.
- **Quantum computing (research stage):** providers already offer access to quantum hardware as a cloud service; how it will interact with virtualization is an open research question.

Real-World Applications

Table 1.28 Real-world applications of virtualization

Application	Benefits
Data-centre consolidation	Fewer physical servers, lower power and cooling cost, faster provisioning.
Virtual desktop infrastructure (VDI)	Central desktop management, data kept off endpoints, support for remote work.
Healthcare	Secure hosting and sharing of patient records; rapid scaling of telemedicine platforms.
Financial services	Isolated environments for trading and compliance testing; robust DR for core banking.
Education	Virtual labs that give every student an identical environment on any device.

Virtualization has grown from a mainframe time-sharing technique into the base layer of every cloud. Its core ideas (abstraction, isolation, pooling and mobility of workloads) reappear in every later section of this chapter.

1.4 Global Cloud Infrastructure

A cloud provider's promise of availability and low latency rests on physical facilities spread across the world. We use Amazon Web Services as the reference example in this section; Azure and Google Cloud use very similar building blocks, and Section 1.10 maps the terms between them.

The **AWS Global Infrastructure** is a worldwide network of data centres, organized into Regions and Availability Zones and extended by edge locations. As of early 2026, AWS reported 120 Availability Zones in 38 geographic Regions, with further Regions announced, plus the AWS European Sovereign Cloud launched in January 2026 as a separate partition in Germany. These figures change several times a year, so the AWS Global Infrastructure web page should be checked for current numbers. India is served by two AWS Regions: Asia Pacific (Mumbai), `ap-south-1`, and Asia Pacific (Hyderabad), `ap-south-2`.

> **REAL-WORLD ANALOGY: A RAILWAY NETWORK**
>
> A Region is a city with several independent railway stations (Availability Zones). Each station has its own power supply and staff, so a fire in one station does not stop trains at the others, yet the stations are close enough to be linked by fast shuttle trains. Edge locations are small ticket counters spread across many towns so that passengers can buy tickets without travelling to the city.

Why does this structure matter? AWS offers more than 200 fully featured services to organizations ranging from start-ups to governments. Those customers need to deploy applications close to their users for low latency, keep copies in separate locations for fault tolerance, and keep data inside specific countries for legal reasons. The global infrastructure is designed to make all three possible.

1.4.1 Components of the Global Infrastructure

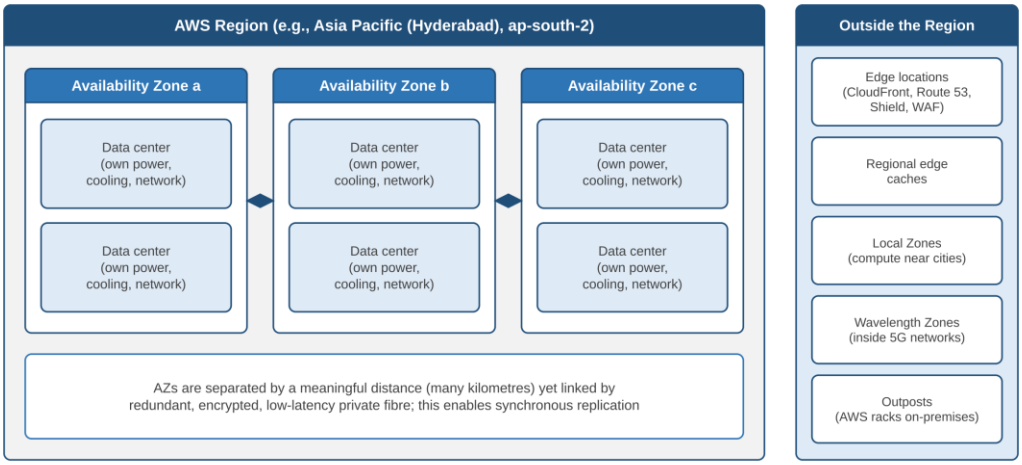


Figure 1.16 Hierarchy of AWS global infrastructure: a Region contains several Availability Zones, each made of one or more data centres; edge services and extensions sit outside the Region.

Data Centre

A **data centre** is a physical facility that houses servers, storage systems and networking equipment, with redundant power, cooling and physical security. Customers never choose an individual AWS data centre; they choose an Availability Zone, and AWS decides which facility inside it hosts their resources. Running an application across several data centres improves availability: if one fails, workloads continue in another.

Availability Zone (AZ)

An **Availability Zone** is one or more discrete data centres within a Region, each with redundant power, networking and connectivity. AZs in a Region are physically separated by a meaningful distance, so that floods, fires or power failures are unlikely to affect more than one, yet they are connected by redundant, encrypted, low-latency, high-throughput private fibre. AWS builds each new Region with at least three AZs. Many AWS services, such as Amazon S3 and Amazon RDS Multi-AZ, replicate data across AZs automatically to protect against failures.

Region

A **Region** is a geographic area that contains a cluster of Availability Zones. Regions are isolated from each other: a problem in one Region should not spread to another, and data stays in the Region where the customer puts it unless the customer moves it. In the AWS Management Console, CLI or SDK, the customer selects a target Region (for example `ap-south-1`). For global services such as AWS Identity and Access Management (IAM), the console's Region selector shows “Global”.

Table 1.29 Criteria for selecting an AWS Region

Criterion	Question to ask	Example
Proximity (latency)	Where are most users?	Users in Telangana are served with lower latency from Hyderabad (ap-south-2) than from Ireland.
Compliance and data residency	Must data stay in a country?	Regulations may require Indian customer data to remain in India; EU data under GDPR is often kept in EU Regions.
Service availability	Is the needed service offered there?	New services often launch first in large Regions such as us-east-1.
Cost	What is the price in that Region?	Prices differ by Region; us-east-1 is often cheaper than many Asia Pacific Regions.
Resilience	Is a second Region needed for disaster recovery?	Primary in Mumbai, recovery in Hyderabad.

Listing 1.8 Listing Regions and Availability Zones with the AWS CLI

```
# All Regions enabled for this account
aws ec2 describe-regions --query "Regions[].RegionName" --output table

# Availability Zones in the Hyderabad Region
aws ec2 describe-availability-zones --region ap-south-2 \
    --query "AvailabilityZones[].[ZoneName,ZoneId,State]" --output table
```

Note that an AZ name such as `ap-south-2a` is mapped to a physical zone independently for each AWS account. The **AZ ID** (for example `aps2-az1`) identifies the same physical zone across accounts and should be used when two accounts must coordinate placement.

Points of Presence: Edge Locations and Regional Edge Caches

Outside the Regions, AWS operates a globally distributed network of **Points of Presence (PoPs)**, made up of edge locations and regional edge caches. They serve as the front door for edge services such as Amazon CloudFront (content delivery network), Amazon Route 53 (DNS) and AWS Global Accelerator.

- **Edge locations** cache content close to end users. If a requested object is already cached, it is served immediately without contacting the origin server. Lambda@Edge and CloudFront Functions can run code at these locations.
- **Regional edge caches** sit between edge locations and the origin. They have larger caches and hold objects that are not popular enough to stay at every edge location but are still requested often enough to keep close to users, reducing load on the origin.
- **Security at the edge:** PoPs host AWS Shield (DDoS protection) and AWS WAF (web application firewall), which filter malicious traffic before it reaches the customer's virtual private cloud (VPC).

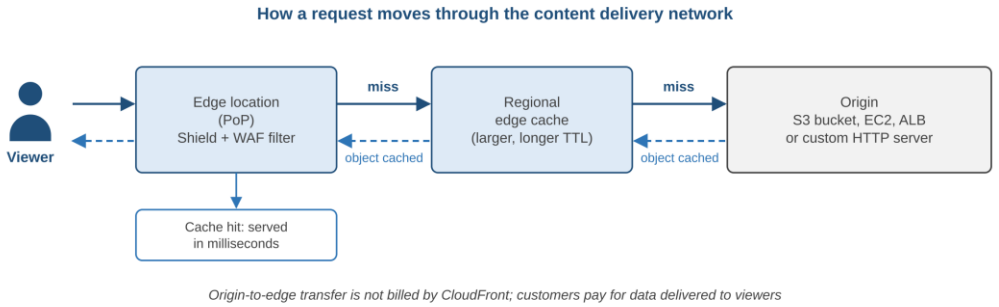


Figure 1.17 Request path through Amazon CloudFront: edge location, regional edge cache and origin.

CloudFront follows a pay-as-you-go model. Data transferred from AWS origins such as Amazon S3, Amazon EC2 or Elastic Load Balancing to CloudFront edge locations is not charged; customers pay for data delivered from the edge to viewers and for requests. Serving cached content from the nearest edge location therefore reduces both latency and origin cost.

Infrastructure Extensions

- **Local Zones** place compute, storage and database services close to large population or industry centres for single-digit-millisecond latency.
- **Wavelength Zones** embed AWS compute and storage inside telecommunications providers' 5G networks for ultra-low-latency mobile applications.
- **AWS Outposts** delivers AWS-designed racks and servers to a customer's own data centre, running the same APIs and tools as the Region, for workloads that must stay on-premises.

Global, Regional and Zonal Services

Every AWS service has a **scope**, and knowing it is essential for designing resilient systems.

Table 1.30 Scope of common AWS services

Scope	Meaning	Examples
Global	One control plane for the whole account across all Regions.	IAM, Route 53, CloudFront, AWS Organizations
Regional	Resources exist in one Region and are usually replicated across its AZs by the service.	Amazon S3 buckets, DynamoDB tables, Lambda functions, VPCs
Zonal	Resources live in one AZ; the customer must deploy copies in other AZs for resilience.	EC2 instances, EBS volumes, subnets

> **CAUTION: A ZONAL RESOURCE IS A SINGLE POINT OF FAILURE**
> An application running on one EC2 instance with one EBS volume lives in one AZ. If that AZ has a problem, the application is down even though the Region is healthy. Resilience comes from deploying across AZs, as shown in Figure 1.18.

1.4.2 Designing for Availability with Multiple AZs

The standard pattern for a highly available web application is to spread every tier across at least two Availability Zones. Figure 1.18 shows a typical arrangement: DNS directs users to a load balancer that spans both AZs, an Auto Scaling group keeps application instances running in each AZ, and a managed database keeps a synchronously replicated standby in the second AZ that takes over automatically if the primary fails.

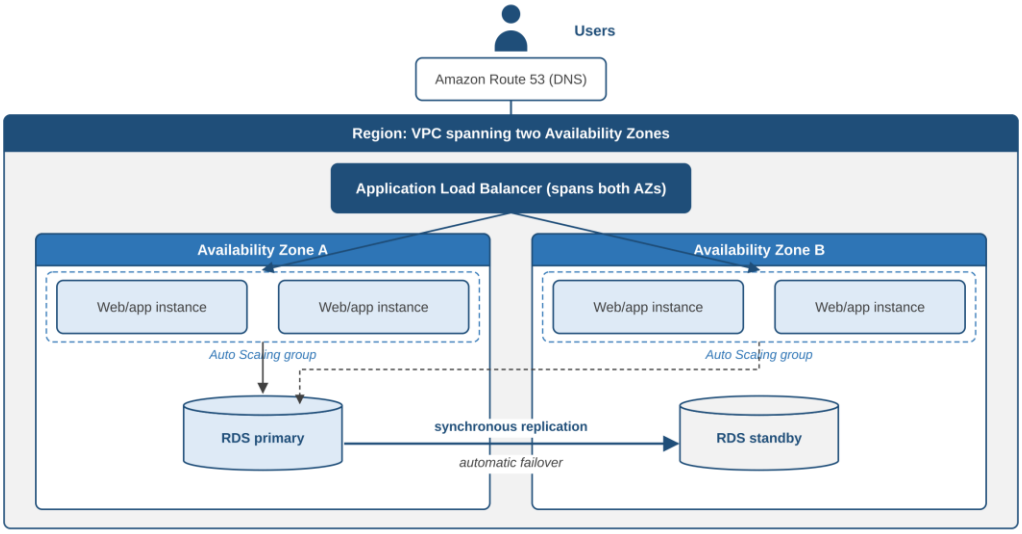


Figure 1.18 Multi-AZ web application. Every tier is present in two Availability Zones; application instances in both AZs write to the primary database.

1.4.3 Benefits of the Global Infrastructure

Table 1.31 Benefits of AWS global infrastructure

Benefit	How it is delivered
Availability	Isolated Regions with multiple AZs; applications spread across AZs survive the loss of a facility; control planes and Regional API endpoints are designed to keep working during disruptions.
Security	Facilities and hardware built to high global security standards; all traffic on the AWS global network between facilities is encrypted at the physical layer before leaving secured facilities; customers control encryption, movement and retention of their data.
Performance	Low latency and low packet loss over a redundant 400 GbE fibre backbone; Local Zones and Wavelength bring infrastructure closer to users and 5G devices.

Benefit	How it is delivered
Flexibility	The same APIs, tools and control plane in Regions, Local Zones, Wavelength and Outposts; sovereign options for regulated customers.
Scalability	Hundreds or thousands of servers can be provisioned in minutes and released when no longer needed, avoiding over-provisioning.

1.5 The AWS Well-Architected Framework

Building a system that works on the first day is not hard. Building one that stays secure, recovers from failure, performs well under load, costs no more than necessary and can be operated calmly at 2 a.m. requires discipline. The **AWS Well-Architected Framework** captures that discipline as a set of pillars, design principles and review questions drawn from thousands of customer architectures.

> **DEFINITION: WELL-ARCHITECTED FRAMEWORK**
> The **AWS Well-Architected Framework** is a set of best practices, design principles and questions that help cloud architects evaluate and improve workloads against six pillars: Operational Excellence, Security, Reliability, Performance Efficiency, Cost Optimization and Sustainability.

AWS published the framework in 2015 with four pillars. Operational Excellence was added in 2016, and Sustainability became the sixth pillar in December 2021. Microsoft (Azure Well-Architected Framework) and Google (Google Cloud Architecture Framework) publish comparable frameworks, so the ideas transfer across providers.

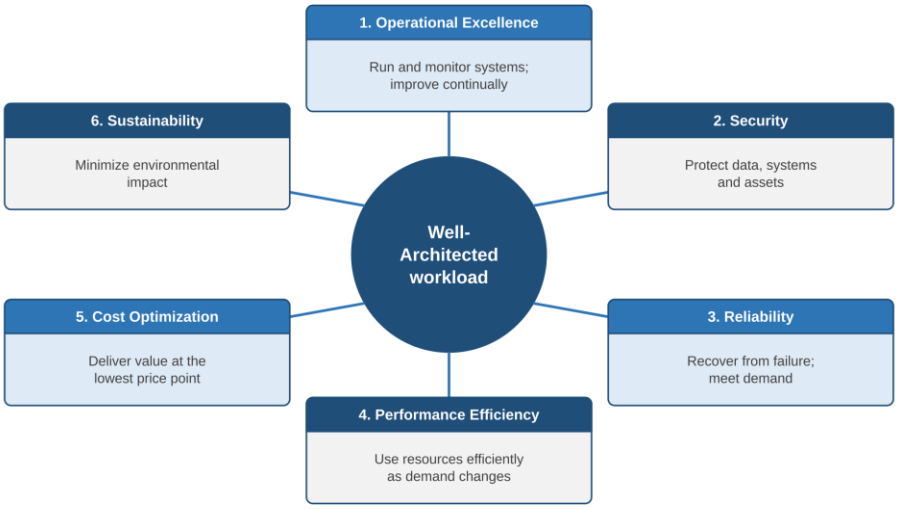


Figure 1.19 The six pillars of the AWS Well-Architected Framework.

General Design Principles

- **Stop guessing capacity needs:** scale with demand instead of buying for a forecast peak.
- **Test systems at production scale:** create a full-size test environment on demand and delete it afterwards.
- **Automate to make experimentation easier:** infrastructure as code makes changes repeatable and reversible.

- **Allow for evolutionary architectures:** design so that components can be replaced as needs change.
- **Drive architectures using data:** base design choices on measurements, not opinion.
- **Improve through game days:** rehearse failures and incidents regularly.

1.5.1 Pillar 1: Operational Excellence

Operational Excellence focuses on running and monitoring systems to deliver business value, and on continually improving the supporting processes and procedures.

- **Perform operations as code:** define infrastructure and runbooks as code so they are versioned and repeatable.
- **Make frequent, small, reversible changes:** limit the impact of each change and allow quick rollback.
- **Refine operations procedures frequently:** update runbooks as the system evolves.
- **Anticipate failure:** run pre-mortems and test failure scenarios.
- **Learn from all operational events:** share lessons from incidents across teams.
- **Implement observability:** collect the metrics, logs and traces needed to understand behaviour.

> **EXAMPLE**
> Netflix popularized **chaos engineering** with tools such as Chaos Monkey, which deliberately terminates production instances. By injecting failures on purpose, engineers find weaknesses before customers do. AWS offers a managed equivalent, AWS Fault Injection Service.

1.5.2 Pillar 2: Security

The Security pillar covers protecting information, systems and assets while delivering business value through risk assessment and mitigation.

- **Implement a strong identity foundation:** least privilege, separation of duties and centralized identity.
- **Maintain traceability:** log and audit every action and change.
- **Apply security at all layers:** network edge, VPC, subnet, load balancer, instance, OS and application.
- **Automate security best practices:** enforce controls through code and policy.
- **Protect data in transit and at rest:** classify data and use encryption, tokenization and access control.
- **Keep people away from data:** reduce direct human access to production data.
- **Prepare for security events:** rehearse incident response.

EXAMPLE

A bank implementing this pillar uses AWS IAM for fine-grained access control, AWS CloudTrail to record every API call, Amazon GuardDuty for threat detection and AWS Key Management Service (KMS) to manage encryption keys.

1.5.3 Pillar 3: Reliability

Reliability is the ability of a workload to perform its intended function correctly and consistently, to recover from infrastructure or service disruptions, to acquire resources to meet demand, and to mitigate disruptions such as misconfiguration or transient network problems.

- **Automatically recover from failure:** monitor key indicators and trigger automated recovery.
- **Test recovery procedures:** simulate failures to validate recovery.
- **Scale horizontally:** replace one large resource with many small ones to reduce the impact of a single failure.
- **Stop guessing capacity:** use auto-scaling to match demand.
- **Manage change through automation:** apply infrastructure changes with code, not by hand.
- **Eliminate single points of failure:** design with redundant components across AZs.

EXAMPLE

A travel-booking platform uses Amazon EC2 Auto Scaling across three AZs so that capacity follows traffic during holiday booking peaks, and a failed instance is replaced automatically within minutes.

1.5.4 Pillar 4: Performance Efficiency

Performance Efficiency is the ability to use computing resources efficiently to meet requirements, and to keep that efficiency as demand changes and technologies evolve.

- **Democratize advanced technologies:** consume complex technologies (machine learning, NoSQL, media transcoding) as managed services.
- **Go global in minutes:** deploy in several Regions to reduce latency for users worldwide.
- **Use serverless architectures:** remove the need to run servers for traditional compute activities.
- **Experiment more often:** compare instance types, storage and configurations cheaply.
- **Consider mechanical sympathy:** choose the technology that best matches the access pattern of the workload.

- **Monitor performance:** use Amazon CloudWatch and AWS X-Ray to find bottlenecks.

> **EXAMPLE**
> A video-streaming service distributes content through Amazon CloudFront, so viewers download from the nearest edge location instead of a single distant origin.

1.5.5 Pillar 5: Cost Optimization

Cost Optimization is the ability to run systems that deliver business value at the lowest price point.

- **Implement cloud financial management:** build the processes and teams (often called FinOps) that own cloud cost.
- **Adopt a consumption model:** pay only for the resources you use, and switch off non-production environments outside working hours.
- **Measure overall efficiency:** relate cost to business output; use AWS Cost Explorer and AWS Budgets to track spend and alert on thresholds.
- **Stop spending money on undifferentiated heavy lifting:** use managed services instead of running databases and servers yourself.
- **Analyse and attribute expenditure:** tag resources so that costs can be assigned to teams and products; use AWS Trusted Advisor to find idle or over-provisioned resources.

1.5.6 Pillar 6: Sustainability

Sustainability focuses on minimizing the environmental impact of running cloud workloads. Amazon set a goal of matching all the electricity used across its operations with renewable energy and reported reaching that goal in 2023.

- **Understand your impact:** measure the environmental footprint of workloads, for example with the AWS Customer Carbon Footprint Tool.
- **Establish sustainability goals:** set targets per workload, such as compute per transaction.
- **Maximize utilization:** right-size resources so that fewer servers do the same work.
- **Anticipate and adopt more efficient hardware and software:** for example, energy-efficient processors such as AWS Graviton.
- **Use managed services:** shared services run at higher utilization than dedicated ones.
- **Reduce the downstream impact:** lower the energy needed by users' devices to use your service.

EXAMPLE

An e-commerce company uses AWS Compute Optimizer to right-size its EC2 instances, removing over-provisioning and lowering energy use. It also schedules batch jobs to run only when needed and lets Auto Scaling shut down idle capacity.

Table 1.32 Summary of the six pillars

Pillar	Central question	Representative AWS services
Operational Excellence	How do we run, observe and improve the workload?	CloudFormation, Systems Manager, CloudWatch
Security	How do we protect data, systems and assets?	IAM, CloudTrail, GuardDuty, KMS, WAF
Reliability	How does the workload recover and meet demand?	Auto Scaling, Multi-AZ RDS, Route 53, AWS Backup
Performance Efficiency	Are we using the right resources efficiently?	CloudFront, Lambda, ElastiCache, X-Ray
Cost Optimization	Are we delivering value at the lowest cost?	Cost Explorer, Budgets, Savings Plans, Trusted Advisor
Sustainability	How do we minimize environmental impact?	Compute Optimizer, Graviton, Carbon Footprint Tool

1.5.7 The Well-Architected Review

The framework is applied through a **Well-Architected Review**, a structured conversation in which architects answer a series of questions about a specific workload, pillar by pillar. The review is not an audit; its purpose is to find risks and agree on improvements. Figure 1.20 shows the cycle.

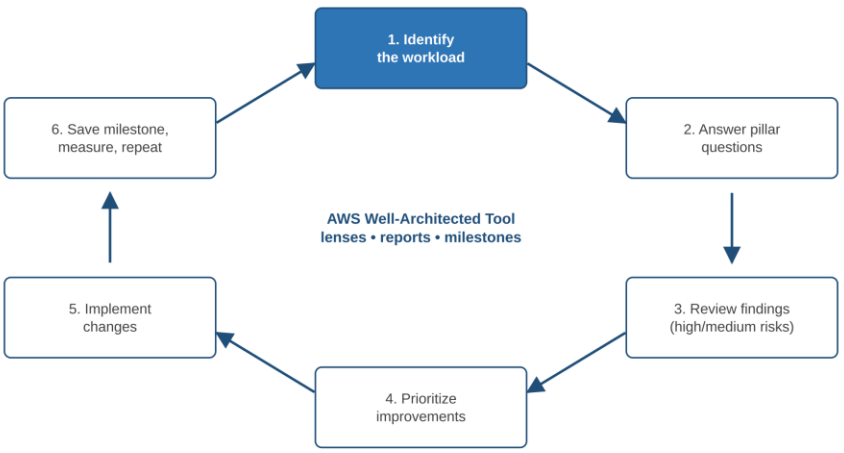


Figure 1.20 The Well-Architected Review cycle.

1. **Identify the workload:** define the application, its owners and its business goals.
2. **Answer the pillar questions:** for example, “How do you detect and investigate security events?”
3. **Review findings:** each answer is compared with best practice, and risks are classified as high risk issues (HRIs) or medium risk issues (MRIs).
4. **Prioritize:** decide which improvements to tackle first based on business impact.
5. **Implement:** carry out the improvement plan.
6. **Save a milestone and repeat:** record the state of the workload and review again after significant changes.

AWS provides the **AWS Well-Architected Tool** in the console at no additional charge. Its main capabilities are:

- Self-service reviews of workloads against current best practices.
- Reports that highlight strengths, risks and an improvement plan.
- Milestones that track the state of a workload over time.
- Risk-based prioritization of improvements.
- **Lenses**, which add questions for specific domains such as Serverless, SaaS, Machine Learning, Data Analytics or Financial Services; organizations can also write custom lenses.
- Sharing of workloads with team members and AWS Partners for collaborative review.
- Links to whitepapers, documentation and videos for each best practice.
- An API for integrating reviews into existing workflows.

1.6 Migration Strategies: The 7 Rs

Most organizations do not start in the cloud. They own tens or hundreds of applications running in their own data centres, and they must decide what to do with each one. A **migration strategy** is the plan for moving software, data and infrastructure from their current location, such as an on-premises server room, to a cloud platform such as AWS. The decision is made per application, not once for the whole company.

> **REAL-WORLD ANALOGY: MOVING HOUSE**
> When a family moves to a new city, they do not treat every possession the same way. Old newspapers are thrown away (**retire**). The piano stays with a relative for now (**retain**). Most furniture goes into the truck unchanged (**rehost**). The whole shipping container is moved as a unit (**relocate**). The old gas stove is swapped for the induction plate the new flat needs (**replatform**). The heavy desktop computer is replaced with a laptop subscription (**repurchase**). The children's room is redesigned completely for the new space (**refactor**).

Gartner described five ways to migrate applications in 2010. AWS later expanded the list into the “6 Rs” and then the **7 Rs** by adding Relocate. The seven strategies are, in order of increasing change: Retire, Retain, Rehost, Relocate, Replatform, Repurchase and Refactor. Figure 1.21 groups them by how much the application changes.

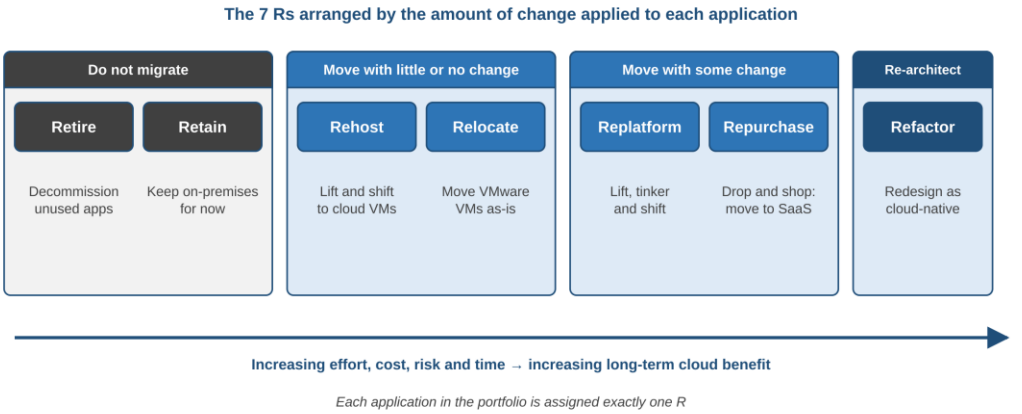


Figure 1.21 The 7 Rs arranged by the amount of change applied to each application.

Cloud migration skills are in high demand because almost every organization is somewhere on this journey. Understanding the 7 Rs helps architects build credible migration plans, helps businesses adapt quickly and control cost, and enables teams to modernize applications in steps rather than in one risky leap.

1.6.1 Retire

Retiring means decommissioning applications that are no longer useful. Portfolio discovery regularly finds applications with no active users, duplicates of other systems, or

functions absorbed elsewhere. Shutting them down removes licence, hardware and maintenance costs and shrinks the security attack surface.

> **EXAMPLE**
> An old intranet reporting tool is found to have had five logins in a year, all from the same administrator. Its data is archived to low-cost storage such as Amazon S3 Glacier, and the application and its two servers are switched off.

1.6.2 Retain

Retaining (sometimes called revisiting) means keeping an application in its current environment for now. Reasons include regulatory requirements, very low latency needs, a recent hardware investment, a planned replacement, or dependencies that are not yet ready to move. Retained applications are reviewed again later.

> **EXAMPLE**
> A factory's machine-control system must respond in under a millisecond and is certified with specific hardware. It stays on-premises and is connected to the cloud only for reporting.

1.6.3 Rehost (Lift and Shift)

Rehosting moves an application to the cloud without changing its architecture or code: the servers are copied to cloud virtual machines. It is the fastest way to exit a data centre and is often the first step before later optimization. AWS Application Migration Service (AWS MGN) replicates source servers continuously into AWS and launches them as EC2 instances with minimal downtime; AWS Migration Hub tracks progress.

> **EXAMPLE**
> A company moving out of a leased data centre within six months rehosts 150 Windows and Linux servers to Amazon EC2 using AWS MGN, then optimizes them after the deadline.

1.6.4 Relocate (Hypervisor-Level Lift and Shift)

Relocating moves a large number of virtual machines from an on-premises virtualization platform to a cloud-hosted version of the same platform, without buying new hardware, rewriting applications or even changing the VMs. The best-known example is moving VMware vSphere workloads to VMware Cloud on AWS using VMware HCX, which supports live migration of running VMs. Because the hypervisor and operations tools stay the same, the team's existing skills still apply.

EXAMPLE

An organization with 800 VMware VMs moves them to VMware Cloud on AWS over a weekend, keeping the same management console and backup tools, before deciding which applications to modernize.

1.6.5 Replatform (Lift, Tinker and Shift)

Replatforming makes a small number of targeted changes so that an application benefits from cloud services, without changing its core architecture. Typical changes are moving a self-managed database to a managed service, moving an application to a managed runtime, or containerizing it.

EXAMPLE

A Java web application keeps its code unchanged, but its self-managed MySQL server is replaced by Amazon RDS for MySQL. Backups, patching and Multi-AZ failover are now handled by the service.

1.6.6 Repurchase (Drop and Shop)

Repurchasing replaces an existing application with a different product, usually a SaaS offering. The organization stops maintaining its own software and migrates data and users to the new platform.

EXAMPLE

An on-premises CRM system is replaced by Salesforce, and a locally installed HR package is replaced by Workday. The business gains automatic updates and scalability but must migrate data and retrain users.

1.6.7 Refactor (Re-architect)

Refactoring redesigns an application to use cloud-native features, typically to improve scalability, agility, resilience or cost. It is the most expensive and time-consuming strategy, and it is justified when the business needs capabilities the current architecture cannot provide.

EXAMPLE

A monolithic order-management application that cannot cope with festival-season traffic is split into microservices. Some run on AWS Lambda, others in containers on Amazon EKS, and order data moves to Amazon DynamoDB, which scales automatically.

1.6.8 Choosing a Strategy

Table 1.33 summarizes the seven strategies, and Figure 1.22 gives a simple decision sequence. Real portfolio decisions also weigh business priority, risk, team skills and deadlines, so the flow should be treated as a starting point for discussion rather than a rule.

Table 1.33 Summary of the 7 Rs

Strategy	Also called	What changes	Typical tools or targets	Use when
Retire	Decommission	Application is switched off	Archive storage	The application is outdated, unused or too costly to keep.
Retain	Revisit	Nothing, for now	Hybrid connectivity	Compliance, latency or readiness keep it on-premises.
Rehost	Lift and shift	Hosting location only	AWS MGN, Amazon EC2	You need a fast, low-risk move with minimal change.
Relocate	Hypervisor lift and shift	Hosting location of whole VMs	VMware Cloud on AWS, HCX	Large VMware estates must move quickly with existing tools.
Replatform	Lift, tinker and shift	A few components	Amazon RDS, Elastic Beanstalk, containers	You want quick wins such as managed databases without a rewrite.
Repurchase	Drop and shop	The product itself	SaaS (Salesforce, Workday)	A SaaS product meets the need better than maintaining your own.
Refactor	Re-architect	Architecture and code	Lambda, EKS, DynamoDB	Business goals require cloud-native scale, agility or resilience.

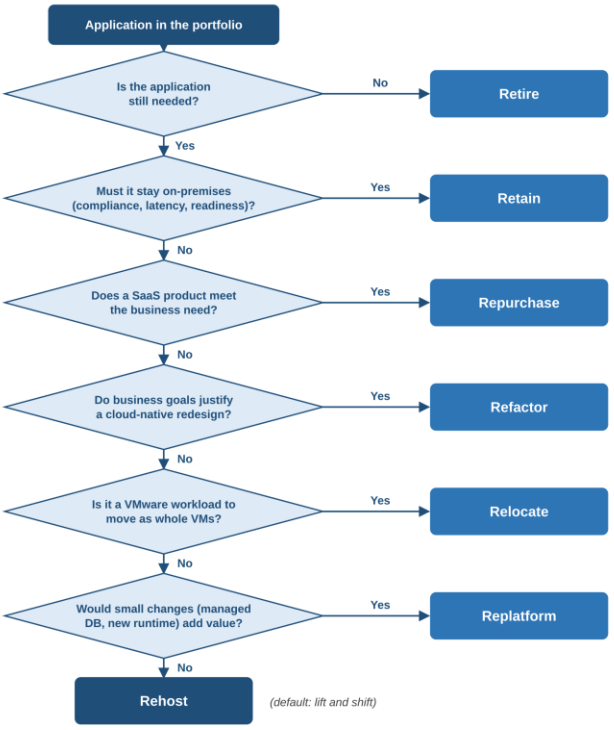


Figure 1.22 A simplified decision sequence for assigning one of the 7 Rs to an application.

1.7 The Cloud Migration Process

Choosing an R for each application is only one part of a migration. The whole effort follows a repeatable process. AWS describes it in three phases (**Assess**, **Mobilize**, and **Migrate and Modernize**), which we expand here into seven practical steps. Figure 1.23 shows how the steps map to the phases.

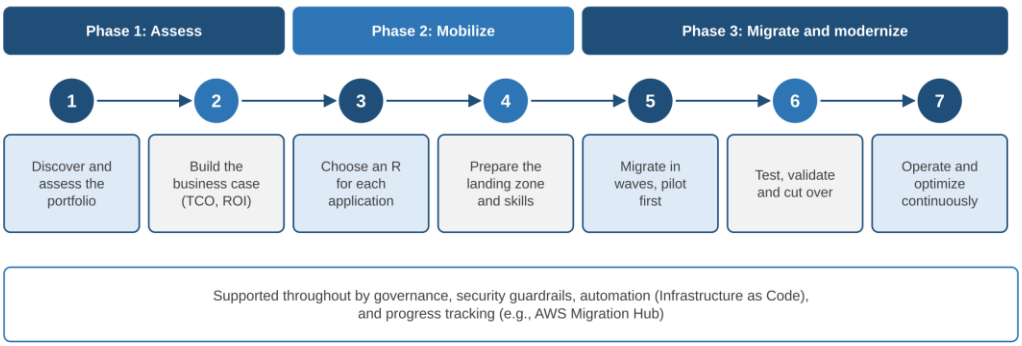


Figure 1.23 Seven steps of cloud migration grouped into the three AWS migration phases.

1. **Discover and assess the portfolio.** Build an inventory of servers, applications, dependencies and utilization, using discovery tools and interviews. Identify which applications talk to which.
2. **Build the business case.** Estimate the total cost of ownership before and after migration (see Section 1.8) and the expected benefits, and obtain sponsorship.
3. **Choose an R for each application.** Apply the 7 Rs to every application and group applications that depend on each other into **migration waves**.
4. **Prepare the landing zone and skills.** Set up a secure, multi-account cloud foundation (for example with AWS Control Tower): accounts, networking, identity, logging and guardrails. Train the team.
5. **Migrate in waves, pilot first.** Start with a low-risk pilot to prove the tooling and process, then move waves in order of dependency.
6. **Test, validate and cut over.** Run functional, performance and security tests, agree go/no-go criteria, switch traffic and keep a rollback plan.
7. **Operate and optimize continuously.** Right-size resources, adopt Savings Plans, apply the Well-Architected Framework and plan the next modernization steps.

1.7.1 Best Practices for Cloud Migration

1. Comprehensive Assessment

Before moving anything, assess the current IT estate thoroughly and decide which of the 7 Rs suits each asset. For example, evaluate each application's data, dependencies and

usage to decide whether it should move as-is, be refactored for cloud-native features, or remain on-premises for a specific reason.

2. Use the Well-Architected Framework

Design the target architecture against the six pillars described in Section 1.5, so that security, reliability and cost controls are built in rather than added later.

3. Deploy Across Multiple Availability Zones

Deploy production workloads across at least two AZs, as in Figure 1.18, so that the loss of one zone does not take the application down.

4. Automate with Infrastructure as Code

Use AWS CloudFormation, Terraform or similar tools to define infrastructure as code, and services such as AWS Elastic Beanstalk or CI/CD pipelines to automate deployment. Automation reduces manual errors and makes environments identical. Listing 1.9 shows a small CloudFormation template.

Listing 1.9 AWS CloudFormation template for an encrypted, versioned, private S3 bucket

```
AWSTemplateFormatVersion: '2010-09-09'
Description: Encrypted, versioned bucket for migration artefacts
Resources:
  MigrationArtifactsBucket:
    Type: AWS::S3::Bucket
    Properties:
      VersioningConfiguration:
        Status: Enabled
      BucketEncryption:
        ServerSideEncryptionConfiguration:
          - ServerSideEncryptionByDefault:
              SSEAlgorithm: aws:kms
      PublicAccessBlockConfiguration:
        BlockPublicAcls: true
        BlockPublicPolicy: true
        IgnorePublicAcls: true
        RestrictPublicBuckets: true
Outputs:
  BucketName:
    Value: !Ref MigrationArtifactsBucket
```

5. Security at Every Layer

Use AWS IAM for fine-grained permissions, Amazon VPC for network isolation with private subnets and security groups, and AWS KMS for encryption keys. Apply least privilege from the first day of the landing zone rather than after go-live.

6. Performance Monitoring

Monitor applications and infrastructure with Amazon CloudWatch for metrics, logs and alarms, and AWS X-Ray or OpenTelemetry-based tools for tracing distributed requests (see Section 1.9). Compare post-migration performance with the baseline measured during assessment.

7. Plan for Rollback and Cost Control

Every cut-over needs a tested rollback path. Set budgets and cost alerts before the first wave, because forgotten test resources and data transfer during migration are common sources of unexpected bills.

1.8 Total Cost of Ownership (TCO)

Migration decisions are ultimately financial as well as technical. An organization needs to know whether moving to the cloud will cost more or less than staying where it is, over a realistic time horizon, and including costs that do not appear on an invoice.

> **DEFINITION: CLOUD TOTAL COST OF OWNERSHIP**
>
> **Cloud total cost of ownership (TCO)** is the sum of all costs of acquiring, provisioning, migrating to, operating and eventually exiting a cloud environment over a defined period. A TCO analysis compares this with the full cost of the current or alternative environment (usually on-premises) over the same period.

The difficulty is that TCO is multifaceted. It goes far beyond comparing the price of a server with the price of an instance. Costs fall into two categories. **Direct (tangible) costs** are visible and easy to include: hardware, subscriptions, licences, power. **Indirect (intangible or hidden) costs** are harder to pin down: financial losses during downtime, slower delivery, security risks, staff time spent on maintenance, and the cost of *not* moving. Figure 1.24 shows both groups.

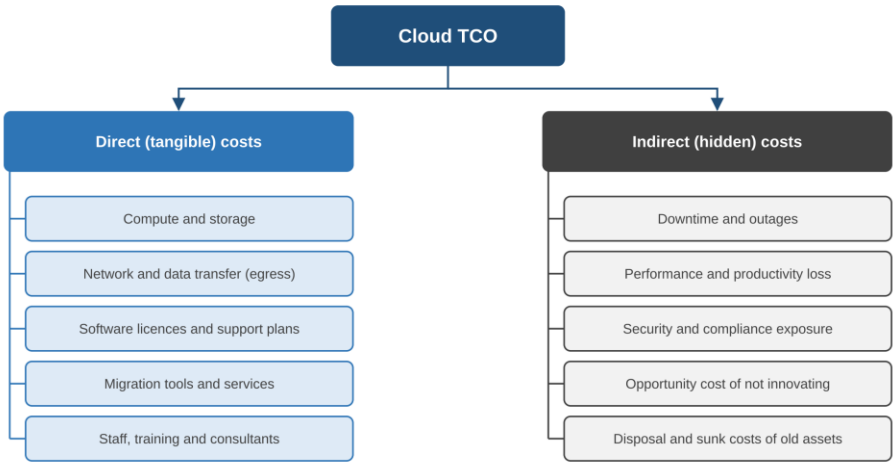


Figure 1.24 Direct and indirect components of cloud TCO.

1.8.1 TCO Versus ROI

TCO and **return on investment (ROI)** are both used in decision making, but they answer different questions.

Table 1.34 TCO compared with ROI

Aspect	Total cost of ownership	Return on investment
Question answered	What will this cost in total over its life?	How much value does this generate relative to what we spend?

Aspect	Total cost of ownership	Return on investment
Focus	Costs, including hidden and opportunity costs	Benefits relative to investment
Typical formula	Sum of all direct and indirect costs over the period	ROI = (Net benefit − Investment) ÷ Investment × 100%
Used to	Compare alternatives (cloud, on-premises, providers)	Judge whether an investment is worthwhile

1.8.2 Why Cloud TCO Analysis Matters

- **Improves decision making.** With public, private, hybrid and multi-cloud options available, concrete numbers are needed to choose a strategy that fits both objectives and budget. The analysis also gives a reference model for future decisions, such as evaluating a new product before investing in it.
- **Increases operational efficiency.** Building a TCO model forces a close look at current usage and usually uncovers idle instances, under-used servers and over-provisioned storage.
- **Supports planning and transparency.** Combining historical and current data allows spending to be forecast. Breaking the total into compute, licences, storage and other components makes it clear what the organization is paying for, and lets costs be attributed to departments, which encourages accountability (the FinOps practice).
- **Evaluates return on investment.** TCO provides the cost side of an ROI calculation and helps ensure that cloud spending does not exceed the value it creates.
- **Strengthens risk management.** A detailed model exposes anomalies and inefficiencies before they grow into serious overruns.
- **Prepares for negotiations.** An organization that knows its numbers negotiates pricing, discounts and contract terms with providers from a position of strength.

1.8.3 Factors That Affect Cloud TCO

Table 1.35 Factors that influence cloud TCO

Factor	Why it matters
Business size, model and domain	Larger enterprises need more complex storage, compute, networking and customization. Regulated sectors such as healthcare and energy need advanced security tools; data residency laws such as GDPR may force multi-Region or multi-cloud designs.
Skills and training	Migration needs cloud specialists, and the new environment needs ongoing monitoring and maintenance by in-house staff or partners.
Pricing model	On-demand is flexible but most expensive per hour. Reserved Instances and Savings Plans give large discounts for one- or three-year commitments; Spot capacity is cheaper still for interruptible work. Models can be combined.

Factor	Why it matters
Migration costs	Set-up, integration, data transfer, consultants, licences, possible downtime, and any refactoring or re-architecture.
Initial investment	On-premises infrastructure needs large capital expenditure up front; cloud spreads cost over time.
Operating costs	Ongoing compute, storage, networking, support plans and maintenance.
Disposal and sunk costs	Legacy hardware may need certified disposal; equipment and licences bought earlier may have little resale value.
Type of migration	Rehosting and replatforming are cheapest to execute; refactoring costs most; repurchasing SaaS can sometimes be cheaper than rebuilding.
Security tooling	Multi-factor authentication, encryption, posture management and regular audits add cost but reduce risk.
Geography of infrastructure	Serving distant users may require several Regions, adding cross-Region data transfer, duplicated storage and extra instances.

1.8.4 Calculating Cloud TCO

A full TCO model can involve dozens of inputs, but every model follows the same five steps, shown in Figure 1.25.

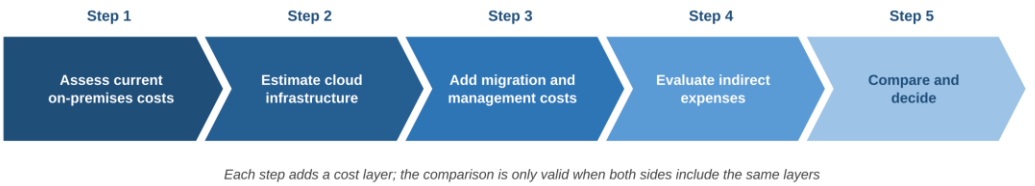


Figure 1.25 Five steps of a cloud TCO calculation.

Step 1: Assess Current On-Premises Costs

- **Hardware and equipment:** servers, storage devices, networking equipment, cooling and spare parts.
- **Data-centre facilities:** racks, floor space, power distribution, fire suppression, physical security.
- **Software:** operating systems, applications, backup and monitoring tools, licence and subscription fees.
- **Human resources:** staff who run networks, databases and infrastructure.
- **Maintenance:** servicing, upgrades, patches and hardware refresh cycles.
- **Security:** network, physical and data protection controls.

This step captures mainly tangible costs. Hidden costs are added in Step 4.

Step 2: Estimate Cloud Infrastructure and Services

- **Services:** which compute, storage, database and network services the workloads need.
- **Workloads:** which applications will run, their technical requirements and priorities; batch jobs are light, while ERP systems are heavy.
- **Resources:** how many and which resources are needed, including capacity for disaster recovery.
- **Pricing model:** on-demand, reserved, Savings Plans or Spot. Provider calculators such as the AWS Pricing Calculator produce detailed estimates.

Step 3: Add Migration and Management Costs

Migration cost depends heavily on the strategy chosen (see Section 1.6). Rehosting is the cheapest; replatforming and refactoring are progressively more expensive; replacing the application entirely can be the costliest. Also include consultation and training for the team, and sufficient network bandwidth for data transfer during migration.

Step 4: Evaluate Indirect Expenses

- **Downtime:** outages or security breaches cause direct financial loss and reputational damage.
- **Slow performance:** bandwidth limits and bottlenecks reduce productivity and revenue.
- **Staff time:** traditional environments consume engineers' time on updates and troubleshooting instead of innovation.
- **Innovation and opportunity:** on-demand access to advanced services is an intangible gain of moving.
- **Elasticity:** the ability to add capacity temporarily during peaks and release it afterwards avoids buying for the peak.

Step 5: Compare On-Premises and Cloud TCO

Finally, compare the totals over the same period with the same cost layers on both sides. Cloud services usually have far lower upfront costs, which lowers the entry barrier for smaller organizations, but for steady 24 x 7 workloads they may cost more over several years. On-premises infrastructure can be cheaper per month once bought, but it does not provide on-demand services that help teams innovate and deploy more often.

1.8.5 Worked Example: Cloud Versus On-Premises Breakeven

We now compare a high-end GPU server in the cloud with an equivalent purchased server, using illustrative list prices. The cloud option is an Amazon EC2 P5 instance with eight NVIDIA H100 GPUs. The on-premises option is a Lenovo ThinkSystem SR675 V3 with comparable GPUs. The prices below were used to illustrate the method; cloud GPU prices change frequently (AWS reduced P5 prices substantially in 2025), so current prices should always be taken from the provider before making a real decision.

Table 1.36 Input values for the worked example (illustrative)

Item	Value
Cloud, on-demand	US$ 98.32 per hour
Cloud, 1-year reserved (about 21% saving)	US$ 77.43 per hour
Cloud, 3-year reserved (about 45% saving)	US$ 53.95 per hour
On-premises server purchase	US$ 833,806 (one-time)
On-premises power	US$ 0.87 per hour
Hours in five years (24 x 365 x 5)	43,800 hours

Five-Year Cost

For continuous use over five years we multiply each hourly rate by 43,800 hours. For the on-premises server we add the purchase price to the power cost.

On-premises 5-year cost = 833,806 + (0.87 × 43,800) = 833,806 + 38,106 = US$ 871,912

Table 1.37 Five-year cost of continuous use

Option	Calculation	Five-year cost (US$)
Cloud on-demand	98.32 × 43,800	4,306,416
Cloud 1-year reserved (renewed)	77.43 × 43,800	3,391,434
Cloud 3-year reserved (renewed)	53.95 × 43,800	2,363,010
On-premises (server + power only)	833,806 + 0.87 × 43,800	871,912

Breakeven Analysis

The **breakeven point** is the number of hours of use at which the cumulative costs of the two options are equal. Let x be the number of hours of use, r the cloud hourly rate, C the on-premises capital cost and c the on-premises hourly running cost:

Cloud cost = $r \cdot x$ On-premises cost = $C + c \cdot x$

Breakeven when $r \cdot x = C + c \cdot x$, so $x^* = C \div (r - c)$

For on-demand pricing:

$x^* = 833{,}806 \div (98.32 - 0.87) = 833{,}806 \div 97.45 \approx 8{,}556$ hours

At 730 hours per month, 8,556 hours is about **11.7 months** of continuous use. Before that point the cloud is cheaper; after it, the purchased server is cheaper. Repeating the calculation for reserved pricing gives about 10,891 hours (14.9 months) for the 1-year rate and 15,708 hours (21.5 months) for the 3-year rate. Figure 1.26 plots the cumulative costs.

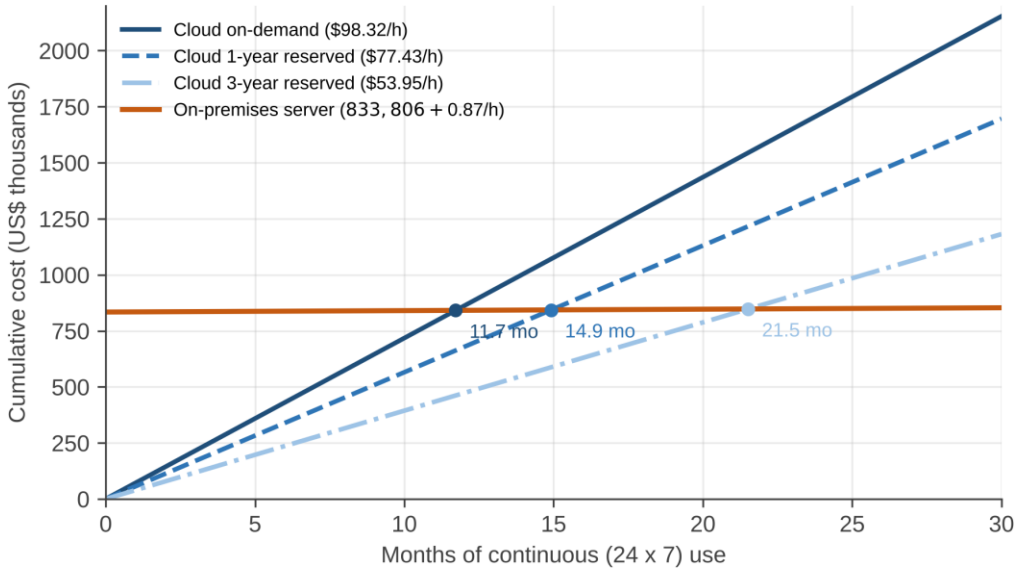


Figure 1.26 Cumulative cost of cloud and on-premises options; dots mark the breakeven points.

KEY POINT: UTILIZATION CHANGES THE ANSWER

The example assumes the server is busy 24 hours a day. Suppose instead the GPUs are needed only 30% of the time. The cloud bill falls to 98.32 × 0.30 ≈ US$ 29.50 per calendar hour, because idle instances can be stopped, while the purchased server costs the same whether it is busy or not. The breakeven moves to 833,806 ÷ (29.50 − 0.87) ≈ 29,100 hours, about **40 months**. Low or uncertain utilization favours the cloud; steady, high utilization favours ownership.

CAUTION: KEEP THE COMPARISON FAIR

This simplified example omits many on-premises costs (data-centre space, cooling, networking, staff, software, maintenance contracts, security, disaster recovery and hardware refresh) and some cloud costs (storage, data egress, support plans). A real TCO model must include the same layers on both sides, as described in Steps 1 to 4.

1.8.6 Reducing Cloud TCO

Cloud is not automatically cheaper than on-premises, as the worked example shows. It becomes cost-effective when it is managed deliberately. Common practices include:

- **Negotiate contracts:** use the TCO model to discuss enterprise discounts and committed-spend agreements with the provider.
- **Use commitment discounts:** Reserved Instances and Savings Plans for steady workloads; Spot capacity for fault-tolerant batch work.

- **Right-size and schedule:** match instance sizes to measured use and stop non-production environments outside working hours.
- **Manage storage lifecycle:** move rarely used data to cheaper tiers automatically and delete what is no longer needed.
- **Automate:** auto-scaling, automated provisioning and clean-up of temporary resources.
- **Practise FinOps:** make teams accountable for their own cloud spend through tagging, showback and budgets.
- **Get expert help when needed:** independent cost reviews or the provider's own tools (AWS Trusted Advisor, Compute Optimizer) often find savings quickly.

Migrating to the cloud without a thorough cost analysis is a risk. A careful TCO and breakeven model turns the decision into an informed one.

1.9 Tracing One HTTP Request Through a Multi-Tier System

Modern cloud applications are built from many tiers and microservices. A single click by a user may pass through a web browser, DNS, a content delivery network, an API gateway, a load balancer, a service mesh, several application services and more than one database before a response comes back. When that click is slow or fails, engineers need to know exactly where the time went and which component returned the error. This section follows one request through every layer and introduces the technique that makes the journey visible.

> **DEFINITION: DISTRIBUTED TRACING**
>
> **Distributed tracing** is an observability technique that records the complete path of a single request as it crosses services and infrastructure components. It shows where the request started and ended, how long each step took, which service caused delays and where errors occurred.

> **REAL-WORLD ANALOGY: TRACKING A COURIER PARCEL**
>
> When you send a parcel, it receives one tracking number. Every hub that handles it scans that number and records the time. If the parcel is late, the tracking history shows that it waited two days at one particular hub. In distributed tracing, the **trace ID** is the tracking number, each service is a hub, and each scan is a **span**.

Consider a customer placing an online order. The request may pass through an Order service, a Payment service, an Inventory service and a database. Distributed tracing links these operations using a common Trace ID, so that they can be viewed as one story.

1.9.1 Architecture of the End-to-End Request Flow

Figure 1.27 shows a conceptual multi-tier architecture and the telemetry pipeline that collects tracing data. Actual deployments may use different routing paths, parallel service calls and asynchronous operations.

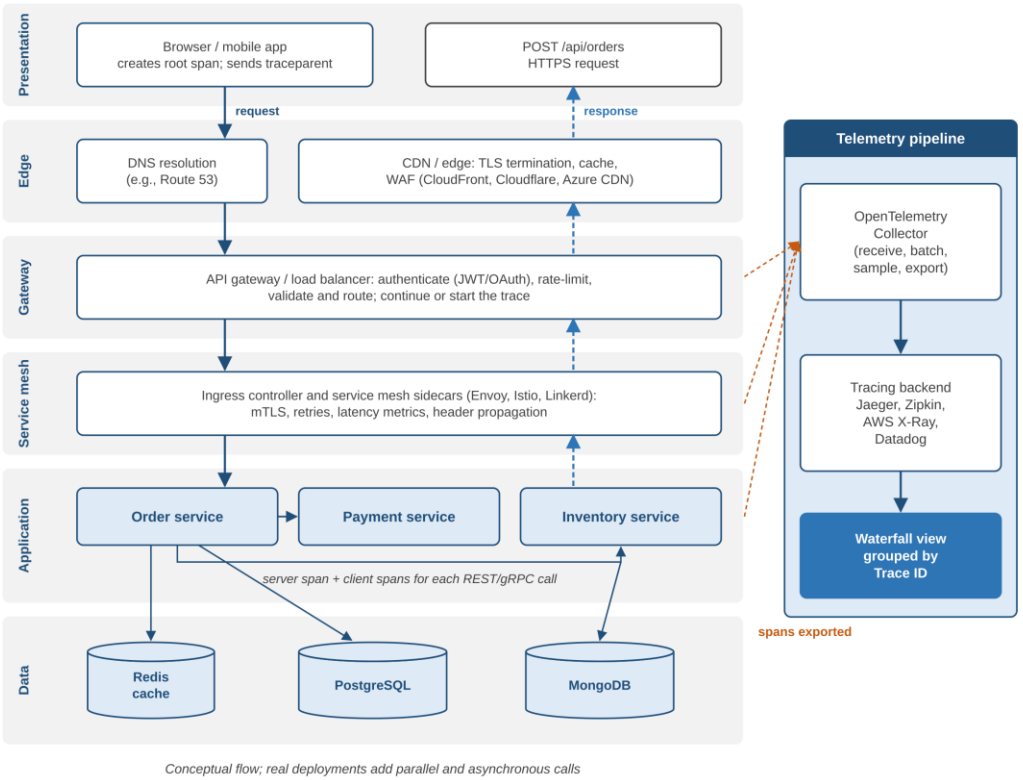


Figure 1.27 End-to-end flow of an HTTP request through the tiers of a cloud application, with spans exported to a tracing backend.

1.9.2 The Request Journey, Tier by Tier

Client (Presentation) Tier

The process begins when a user interacts with a web page or mobile application, and the client sends an HTTPS request. If front-end tracing is enabled, the client creates a root Trace ID and a client span, and passes the trace context to the next tier in HTTP headers using the **W3C Trace Context** standard. For example, the customer clicks **Place Order** and the browser sends `POST /api/orders`.

Edge Network, DNS and CDN

DNS (for example Amazon Route 53) resolves the domain name to an IP address. A TLS handshake establishes a secure connection, and HTTPS may be terminated at the CDN edge. The CDN (Amazon CloudFront, Cloudflare or Azure Front Door) serves cached content directly or forwards the request to the origin, applying web application firewall rules on the way. Edge logs and the trace headers are passed on.

API Gateway, Reverse Proxy or Load Balancer

The request reaches an API gateway or load balancer such as an AWS Application Load Balancer, Amazon API Gateway, NGINX, Kong or Apigee. This layer authenticates the

caller (for example with a JWT or OAuth 2.0 token), applies rate limits, validates the request and routes it to the right service. It continues the existing trace or starts a new one if none was received, and records its own processing time as a span. Here, `/api/orders` is routed to the Order service.

Service Mesh and Ingress Controller

In Kubernetes-based applications, an ingress controller admits the traffic into the cluster and a **service mesh** (Istio, Linkerd, or Envoy-based meshes) manages communication between microservices. Sidecar proxies enforce mutual TLS (mTLS), apply retries and timeouts, record network latency and failures, and propagate trace headers. The Order service therefore talks to the Payment service over an encrypted, observed connection.

Application (Microservices) Tier

This is the business-logic layer. The Order service creates the order, the Payment service charges the customer, the Inventory service reserves stock and a User service validates the customer. Tracing middleware in each service extracts the incoming trace context and creates a **server span**. When one service calls another through REST or gRPC, it creates a **client span** and injects the trace context into the outgoing request. Listing 1.10 shows how a span is created in code.

Listing 1.10 Creating a custom span with the OpenTelemetry API in Python

```
from opentelemetry import trace

tracer = trace.get_tracer("order-service")

def place_order(order):
    # Child span of the incoming server span; same trace ID, new span ID
    with tracer.start_as_current_span("place_order") as span:
        span.set_attribute("order.item_count", len(order.items))
        charge_payment(order)    # outgoing call carries traceparent
        reserve_stock(order)
```

Data Tier: Databases and Caches

The data tier stores and retrieves information. Redis is commonly used as an in-memory cache, PostgreSQL for relational data and MongoDB for documents. Instrumented database drivers create spans for each operation and can record query duration and connection-pool waiting time. Sensitive data, credentials and full personal records must never be written into trace attributes. In our example, the Order service checks cached product data in Redis and stores the order in PostgreSQL.

Response Propagation and Telemetry Export

After processing, the response travels back through the services, gateway and network to the client. Each component closes its span and records the duration and status, such as `200 OK`, `201 Created` or `500 Internal Server Error`. Spans are exported, usually through an **OpenTelemetry Collector**, to a tracing backend such as Jaeger, Zipkin, AWS X-Ray

or Datadog. The backend groups all spans with the same Trace ID and displays them as a timeline or waterfall.

1.9.3 Core Concepts of Distributed Tracing

Table 1.38 Important concepts used in distributed tracing

Concept	Explanation
Trace	The complete record of one end-to-end request, made up of many spans.
Span	One timed operation within a trace, such as gateway processing or a database query, with a start time, duration, status and attributes.
Trace ID	A unique identifier shared by all spans of the same request.
Span ID	A unique identifier for one span.
Parent span ID	The span that started the current span; used to rebuild the parent-child tree.
Context propagation	Passing trace ID and parent span ID from one component to the next, usually in HTTP headers.
Sampling	Recording only a fraction of traces (for example 10%, or all errors) to control cost and overhead.
Attributes	Key-value metadata on a span, such as `http.response.status_code` or `db.system`.

> **KEY POINT**
> The Trace ID stays the same throughout a distributed trace, while every span has its own Span ID. A trace can contain many spans, including nested operations and parallel branches.

1.9.4 The W3C Trace Context Header

Context is passed between services in the standard `traceparent` header. Listing 1.11 shows a request carrying it, and Figure 1.28 breaks the header into its four fields.

Listing 1.11 An HTTP request carrying W3C trace context

```
POST /api/orders HTTP/1.1
Host: shop.example.com
Content-Type: application/json
traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01
tracestate: vendor1=opaqueValue
```

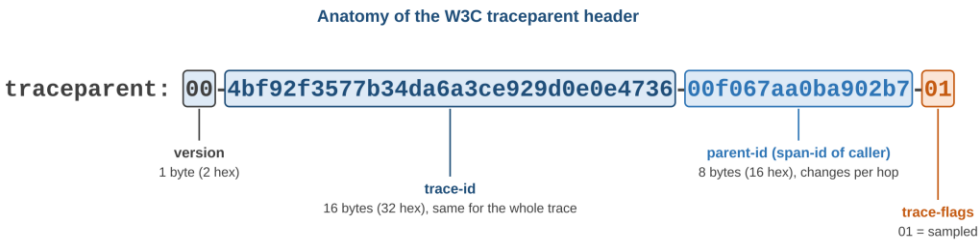


Figure 1.28 The four fields of the traceparent header.

The `traceparent` header carries the version, the trace ID, the caller's span ID (called parent-id in the specification) and trace flags; the flag value `01` means the trace is sampled. The optional `tracestate` header carries vendor-specific tracing information.

1.9.5 Standards and Tools

Table 1.39 Important standards and tools for distributed tracing

Standard or tool	Purpose
W3C Trace Context	Standard for propagating trace information between services using `traceparent` and `tracestate`.
OpenTelemetry	Vendor-neutral CNCF project with APIs, SDKs and a Collector for instrumenting, collecting and exporting traces, metrics and logs.
Jaeger	Open-source distributed tracing backend for storing, searching and visualizing traces.
Zipkin	Open-source distributed tracing system for collecting and viewing trace data.
AWS X-Ray	AWS service for tracing and troubleshooting distributed applications.
Azure Application Insights / Google Cloud Trace	Equivalent managed tracing services on Azure and Google Cloud.
Datadog, New Relic, Dynatrace	Commercial observability platforms that ingest OpenTelemetry data.

1.9.6 Worked Example: An Online Shopping Order

A customer buys a mobile phone from an online store. The request follows these operations:

1. The customer clicks **Place Order**.
2. The API gateway receives and validates the request.
3. The Order service creates a new order.
4. The Payment service verifies the payment.
5. The Inventory service checks product availability.
6. PostgreSQL stores the order information.

7. The response returns to the customer.

The complete request takes 3 seconds. Table 1.40 gives illustrative span timings, and Figure 1.29 shows them as the waterfall a tracing backend would display.

Table 1.40 Illustrative span timings for the order request

Component	Execution time
API gateway	200 ms
Order service	300 ms
Payment service	1,500 ms
Inventory service	400 ms
Database	600 ms
Total (sequential flow)	**3,000 ms**

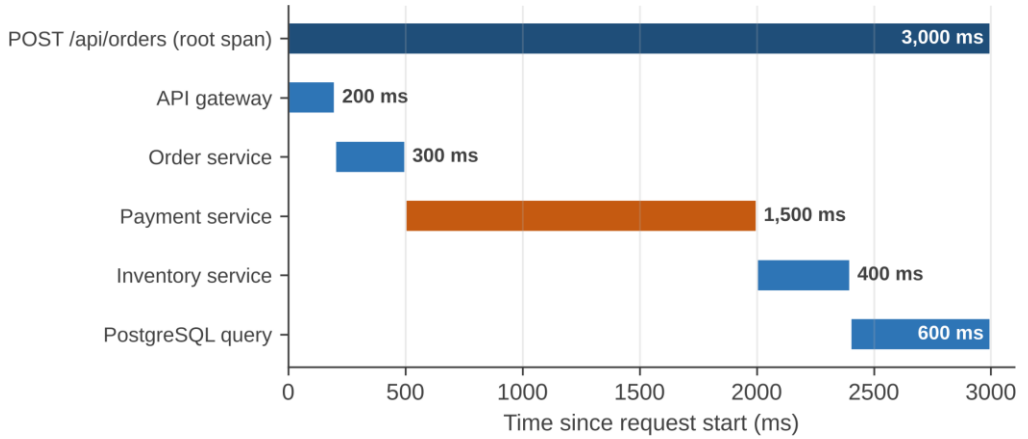


Figure 1.29 Trace waterfall for the order request. The Payment service span dominates the total time.

The Payment service accounts for 1,500 ms, half of the total and the largest single delay. Engineers would investigate the payment provider's API, the network path to it, or the payment database. The total in this example assumes purely sequential processing; real traces often show parallel spans, for example the Payment and Inventory calls running at the same time, which would reduce the overall latency.

1.9.7 Advantages and Good Practices

Table 1.41 Advantages of distributed tracing

Advantage	What it gives the team
End-to-end visibility	The whole path of a request across every participating service.
Performance monitoring	Execution time of each service and operation.

Advantage	What it gives the team
Error detection	The exact service and operation where a failure occurred.
Root cause analysis	Faster location of the source of a problem.
Latency analysis	Slow queries, network delays and bottlenecks made visible.
Dependency mapping	How services call and depend on one another.
Improved reliability	Data for troubleshooting and continuous optimization.

- Instrument with OpenTelemetry so that the backend can be changed without re-instrumenting code.
- Propagate context through every hop, including message queues and asynchronous jobs.
- Use consistent service names and the OpenTelemetry semantic conventions for attributes.
- Sample deliberately: keep all error traces and a representative share of successful ones.
- Never place passwords, tokens or personal data in span attributes.
- Correlate traces with logs and metrics by including the trace ID in every log line.

1.10 Tri-Cloud Terminology Equivalence

Engineers increasingly work across more than one provider. The concepts are largely the same, but the names differ. Table 1.42 maps the most common terms between Amazon Web Services, Microsoft Azure and Google Cloud.

Table 1.42 Equivalent concepts and services across AWS, Azure and Google Cloud

Concept	AWS	Microsoft Azure	Google Cloud
Geographic area	Region	Region (most paired with another Region)	Region
Isolated failure domain	Availability Zone (AZ)	Availability Zone	Zone
Edge and on-premises extensions	Local Zones, Wavelength, Outposts	Azure Extended Zones, Azure Local	Google Distributed Cloud
Virtual machine	Amazon EC2	Azure Virtual Machines	Compute Engine
Object storage	Amazon S3	Azure Blob Storage	Cloud Storage
Block storage	Amazon EBS	Azure Managed Disks	Persistent Disk / Hyperdisk
Virtual network	Amazon VPC	Azure Virtual Network (VNet)	VPC network
Load balancing	Elastic Load Balancing	Azure Load Balancer, Application Gateway	Cloud Load Balancing
DNS	Amazon Route 53	Azure DNS	Cloud DNS
Content delivery	Amazon CloudFront	Azure Front Door	Cloud CDN
Serverless functions	AWS Lambda	Azure Functions	Cloud Run functions
Serverless containers	AWS Fargate, App Runner	Azure Container Apps	Cloud Run
Managed Kubernetes	Amazon EKS	Azure Kubernetes Service (AKS)	Google Kubernetes Engine (GKE)
Managed relational database	Amazon RDS, Aurora	Azure SQL Database, Azure Database for PostgreSQL	Cloud SQL, AlloyDB
Identity and access	AWS IAM, IAM Identity Center	Microsoft Entra ID, Azure RBAC	Cloud IAM
Key management	AWS KMS	Azure Key Vault	Cloud KMS
Monitoring	Amazon CloudWatch	Azure Monitor	Cloud Monitoring
Distributed tracing	AWS X-Ray	Application Insights	Cloud Trace
Infrastructure as code	AWS CloudFormation, CDK	ARM templates, Bicep	Infrastructure Manager (Terraform)

Concept	AWS	Microsoft Azure	Google Cloud
Architecture best practice	AWS Well-Architected Framework	Azure Well-Architected Framework	Google Cloud Architecture Framework

1.10.1 Regions and Zones Compared

- **AWS** groups independent Availability Zones into Regions connected by low-latency private networking. New Regions are built with at least three AZs.
- **Microsoft Azure** uses Regions, many of which are **paired** with another Region in the same geography for disaster recovery and staged platform updates. Availability Zones (physically separate locations with independent power, cooling and networking) are offered in a growing number of Regions, but not in every one.
- **Google Cloud** divides each Region into **zones**, typically three. Zones in a Region are tightly connected low-latency domains suitable for synchronous replication. Google also offers dual-region and multi-region configurations for some storage and database services.

1.10.2 Pricing Calculators and Discount Models

Each provider offers a web-based calculator to model architectures, choose Regions and estimate monthly or annual spend, and each offers discounts in exchange for commitment or flexibility.

Table 1.43 Pricing calculators and discount models

Provider	Calculator	Main discount models
AWS	AWS Pricing Calculator	Savings Plans (1 or 3 years); Reserved Instances (1 or 3 years); Spot Instances for interruptible workloads.
Microsoft Azure	Azure Pricing Calculator	Azure Reservations (1 or 3 years); Azure Savings Plan for Compute; Azure Hybrid Benefit for existing Windows Server and SQL Server licences; Spot VMs.
Google Cloud	Google Cloud Pricing Calculator	Committed use discounts (resource-based or spend-based, with no upfront payment); sustained use discounts applied automatically to eligible VMs; Spot VMs.

1.11 Break-Fix Case Study: Responsibility Misattribution

A **break-fix** scenario is the cycle of something failing in production and a team restoring it. One of the most common and costly root causes of prolonged break-fix incidents in the cloud is **responsibility misattribution**: a customer assumes that the cloud service provider (CSP) handles a security, operational or maintenance task that is in fact the customer's job. The task is never done, something breaks, and the customer waits for a fix that will never come from the provider.

1.11.1 The Shared Responsibility Model

Every major provider publishes a **shared responsibility model** that divides duties between provider and customer. AWS summarises it in two phrases, shown in Figure 1.30:

- **The provider is responsible for security of the cloud:** physical facilities, hardware, the global network and the virtualization layer.
- **The customer is responsible for security in the cloud:** data, identities and access, configuration, operating systems (for IaaS) and applications.

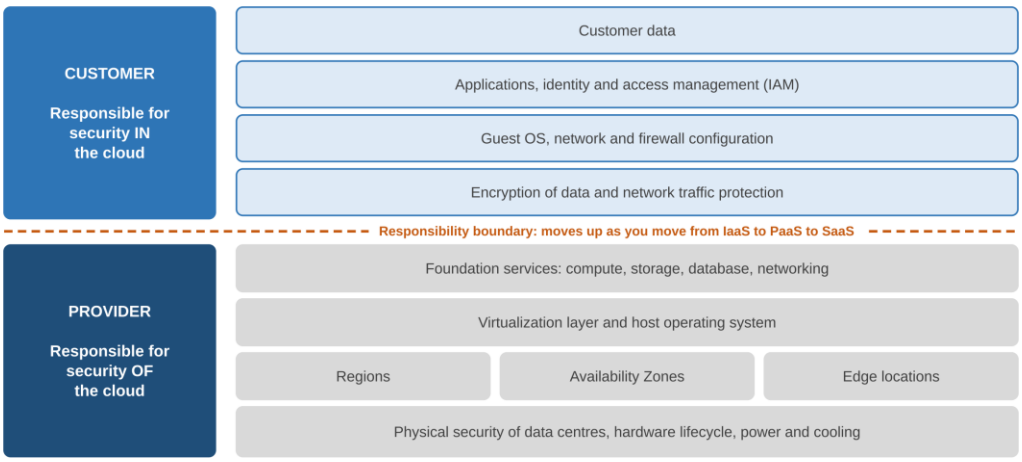


Figure 1.30 The shared responsibility model. The boundary moves upward as the service model becomes more managed.

Azure and Google Cloud publish equivalent models; Google describes its approach as **shared fate**, emphasizing that the provider also helps customers configure their part securely. In every version, the boundary depends on the service model, which is why Figure 1.2 matters so much. Table 1.44 shows the boundary layer by layer.

Table 1.44 Who is responsible for each layer under each service model

Layer	IaaS	PaaS	SaaS
Data and its classification	Customer	Customer	Customer
Identities, accounts, access policies	Customer	Customer	Customer
Application code and configuration	Customer	Customer	Provider (customer configures settings)
Network controls (security groups, firewall rules)	Customer	Shared	Provider
Operating system patching	Customer	Provider	Provider
Runtime and middleware	Customer	Provider	Provider
Virtualization, hosts, physical network	Provider	Provider	Provider
Physical data-centre security	Provider	Provider	Provider

1.11.2 Common Misattribution Scenarios

Table 1.45 Misconceptions and reality by service model

Model and example	The misconception	The reality
IaaS: Amazon EC2, Azure VMs	“The provider hosts the VM, so it will patch the operating system and back up my data.”	The provider guarantees the hardware and hypervisor. If the guest OS is infected with malware or a database is corrupted without backups, recovery is entirely the customer's responsibility.
PaaS: Azure SQL Database, AWS Elastic Beanstalk	“The provider manages the database platform, so my data is automatically secure and private.”	The provider patches the database engine. If the customer sets network access to public or uses weak credentials, the resulting data exposure is the customer's fault.
SaaS: Microsoft 365, Salesforce	“The vendor manages everything. If an employee deletes a critical folder, the vendor will restore it.”	SaaS providers guarantee application availability, not unlimited retention against user error or malicious deletion. Recovery beyond the vendor's retention window needs customer-managed backup.

1.11.3 The Cost of Misattribution

- **Finger-pointing downtime:** hours or days are spent on urgent support tickets, only for the provider to reply that the fault is inside the guest OS or application.
- **The unpatched server:** critical vulnerabilities stay open because the team assumed the provider's maintenance covered their VMs.
- **Data loss:** treating “stored in the cloud” as if it meant “immutably backed up” leads to severe losses during ransomware attacks or accidental deletion.

- **Compliance failures:** auditors hold the customer accountable for controls it never implemented.

1.11.4 Case Study: The Server Nobody Patched

CASE STUDY: AN ILLUSTRATIVE BREAK-FIX INCIDENT

Situation. A retail company rehosts its order-processing application to Amazon EC2 (see Section 1.6). The migration team assumes that, because the servers now run “in AWS”, operating-system patching and backups are handled by the provider. No patch baseline or backup plan is configured.

Break. Eight months later, attackers exploit a publicly known vulnerability in the unpatched operating system and encrypt the application's data volume. The online store stops taking orders.

Misdirected fix. The operations team opens a high-severity support case asking AWS to restore the server. AWS confirms that the underlying host and EBS service are healthy; the compromise is inside the guest OS, which is the customer's responsibility. Because no snapshots exist, there is nothing to restore. The store is offline for two days while the application is rebuilt from source code and partially recovered data.

Root cause. Responsibility misattribution at the IaaS boundary: the guest OS and data layers were nobody's documented responsibility.

Corrective actions. The company adopts AWS Systems Manager Patch Manager with a weekly patch baseline, AWS Backup with daily snapshots copied to a separate account, a responsibility matrix for every workload, and a quarterly restore test.

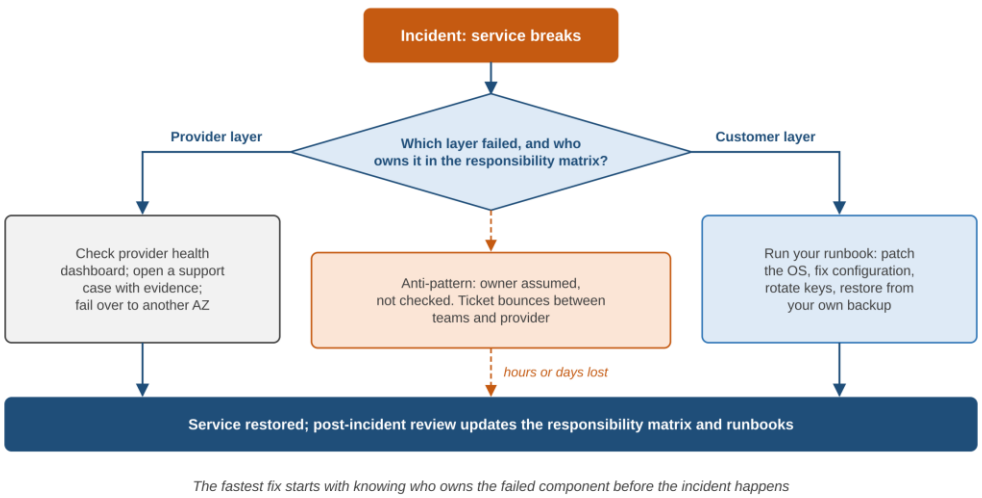


Figure 1.31 Break-fix decision flow. Checking ownership first avoids the anti-pattern in the centre.

1.11.5 Preventing and Fixing Misattribution

1. **Map every asset to a responsibility matrix.** Do not deploy a cloud resource without documenting who configures it, who patches it, who backs it up and who responds when it fails. A simple RACI table is enough (Table 1.46).
2. **Automate configuration with Infrastructure as Code.** Define secure settings once in code so that each new resource inherits them, eliminating manual misconfiguration.
3. **Apply the principle of least privilege.** Grant identities only the permissions they need, so that a mistake or a compromised account cannot change everything.
4. **Use Cloud Security Posture Management (CSPM).** Tools such as AWS Security Hub, Microsoft Defender for Cloud and Google Security Command Center continuously scan for gaps you may have missed, such as unencrypted buckets, open ports or missing backups.
5. **Own backups explicitly.** Configure provider backup services or third-party backup for SaaS data, keep copies in a separate account, and test restores regularly.
6. **Write runbooks and rehearse.** Every runbook should start with “which layer failed, and who owns it?”, as in Figure 1.31.

Table 1.46 Example responsibility matrix for one IaaS workload

Task	Application team	Cloud platform team	Provider
OS patching (monthly baseline)	Consulted	Responsible	Not involved
Data backup and restore testing	Accountable	Responsible	Not involved
Security group and firewall rules	Consulted	Responsible	Not involved
Hypervisor and host hardware	Informed	Informed	Responsible
Application deployment	Responsible	Consulted	Not involved

The shared responsibility model is not a legal footnote. It is an operational map. Teams that read it before deploying, and encode it in their runbooks and automation, resolve incidents in minutes; teams that discover it during an outage lose days.

Chapter Summary

- Cloud computing, as defined by NIST, delivers on-demand, network-accessible, pooled, elastic and metered computing resources.
- Service models define how much of the stack the provider manages: SaaS (complete applications), PaaS (managed platforms), IaaS (virtual infrastructure), and the newer FaaS (event-driven functions) and CaaS (managed container orchestration).
- Deployment models define where resources run and who may use them: public, private, community, hybrid and multi-cloud. Hybrid combines private and public; multi-cloud combines several public providers.
- Virtualization abstracts hardware through hypervisors. Type 1 hypervisors run on bare metal and power data centres; Type 2 hypervisors run on a host OS. Containers virtualize the OS rather than the hardware and are lighter but less isolated than VMs.
- Global infrastructure is organized into Regions, Availability Zones and data centres, extended by edge locations, regional edge caches, Local Zones, Wavelength and Outposts. Resilient designs spread every tier across multiple AZs.
- The AWS Well-Architected Framework evaluates workloads against six pillars: Operational Excellence, Security, Reliability, Performance Efficiency, Cost Optimization and Sustainability, through a repeatable review process.
- The 7 Rs (Retire, Retain, Rehost, Relocate, Replatform, Repurchase, Refactor) assign a migration strategy to each application, within a migration process of assessment, mobilization, and migration and modernization.
- Total cost of ownership includes direct and indirect costs. A breakeven analysis, $x^* = C \div (r - c)$, shows when owning becomes cheaper than renting; utilization strongly affects the result.
- Distributed tracing follows one request through every tier using trace IDs, span IDs and the W3C traceparent header, with OpenTelemetry as the vendor-neutral standard.
- AWS, Azure and Google Cloud offer equivalent services under different names.
- Most prolonged cloud incidents involving security or data loss trace back to misreading the shared responsibility model. Documented ownership, automation, posture management and tested backups prevent them.

Key Terms

Table 1.47 Key terms introduced in this chapter

Term	Definition
Availability Zone (AZ)	One or more isolated data centres within a Region with independent power, cooling and networking.
Breakeven point	The usage level at which the cumulative costs of two options are equal.

Term	Definition
CaaS	Container as a Service: managed container orchestration, usually Kubernetes.
Cold start	Extra latency when a FaaS platform must create a new execution environment.
Community cloud	Cloud infrastructure shared by organizations with common requirements.
Edge location	A point of presence that caches content and runs edge services close to users.
FaaS	Function as a Service: event-triggered code billed per execution.
Hybrid cloud	A combination of private and public cloud environments that are connected and managed together.
Hypervisor	Software that creates, runs and isolates virtual machines.
IaaS	Infrastructure as a Service: virtual machines, storage and networks on demand.
Multi-cloud	Use of two or more public cloud providers.
Multi-tenancy	Many customers sharing one application instance with logical data separation.
PaaS	Platform as a Service: a managed environment to build and run applications.
Region	A geographic area containing multiple Availability Zones.
SaaS	Software as a Service: complete applications delivered over the internet.
Shared responsibility model	The division of security and operational duties between provider and customer.
Span	A single timed operation within a distributed trace.
TCO	Total cost of ownership: all direct and indirect costs over a defined period.
Trace context	Identifiers (trace ID, parent span ID, flags) propagated between services.
Virtual machine (VM)	A software emulation of a computer that runs its own operating system.

Review Questions

A. Multiple-Choice Questions

1. Which NIST characteristic makes pay-per-use billing possible?

(a) Broad network access **(b)** Resource pooling
(c) Measured service **(d)** Rapid elasticity

2. Under IaaS, who is responsible for patching the guest operating system?

(a) The cloud provider **(b)** The customer
(c) The hypervisor vendor **(d)** Nobody; it is automatic

3. A team wants to deploy a web API without managing servers or operating systems, but still write its own code. The best fit is:

(a) SaaS **(b)** PaaS
(c) IaaS **(d)** On-premises

4. Which statement about multi-tenant SaaS is correct?

(a) Each customer has a dedicated database **(b)** Data isolation is physical
(c) Updates are deployed once for all tenants **(d)** Customization is unlimited

5. Cold starts are most closely associated with:

(a) FaaS **(b)** IaaS
(c) Private cloud **(d)** Type 2 hypervisors

6. Using AWS for web hosting and Google Cloud for analytics, with no private data centre, is an example of:

(a) Hybrid cloud **(b)** Community cloud
(c) Multi-cloud **(d)** Private cloud

7. A Type 1 hypervisor:

(a) Runs as an application on Windows **(b)** Runs directly on the hardware
(c) Requires a guest OS to boot **(d)** Is only used on laptops

8. Linux containers isolate processes primarily through:

(a) Binary translation **(b)** Namespaces and control groups
(c) Paravirtualized drivers **(d)** Separate kernels per container

9. Which AWS resource is zonal?

(a) An IAM role **(b)** An S3 bucket
(c) An EBS volume **(d)** A Route 53 hosted zone

10. The Well-Architected pillar added in December 2021 is:

(a) Operational Excellence **(b)** Reliability
(c) Cost Optimization **(d)** Sustainability

11. Moving VMware VMs unchanged to VMware Cloud on AWS is called:

(a) Rehost **(b)** Relocate
(c) Replatform **(d)** Refactor

12. Replacing an on-premises CRM with Salesforce is:

(a) Repurchase **(b)** Retain
(c) Refactor **(d)** Relocate

13. Breakeven hours between a cloud rate r and an owned server with capital cost C and running cost c per hour are:

(a) $C \times (r - c)$ **(b)** $C \div (r + c)$
(c) $C \div (r - c)$ **(d)** $(r - c) \div C$

14. In the traceparent header 00-4bf9...4736-00f0...02b7-01, the final 01 indicates:

(a) Protocol version 1 **(b)** The trace is sampled
(c) One retry attempt **(d)** HTTP status 201

15. According to the shared responsibility model, recovering SaaS data deleted by an employee beyond the vendor's retention period is the responsibility of:

(a) The SaaS vendor **(b)** The underlying IaaS provider
(c) The customer **(d)** The internet service provider

B. Short-Answer Questions

1. State the NIST definition of cloud computing and list its five essential characteristics.
2. Distinguish between a cloud service model and a cloud deployment model with one example of each.
3. Compare single-tenant and multi-tenant SaaS on cost, isolation and upgrade control.
4. Why is FaaS described as “serverless” when servers still exist?
5. Explain how hybrid cloud differs from multi-cloud. Can an organization be both?
6. State the three Popek and Goldberg properties of a virtual machine monitor.
7. Differentiate a Region, an Availability Zone and an edge location.

8. List the six pillars of the AWS Well-Architected Framework with one design principle for each.
9. What is the difference between TCO and ROI?
10. Define trace, span, trace ID and parent span ID.

C. Long-Answer Questions

1. Using a diagram of the technology stack, explain how management responsibility changes across on-premises, IaaS, PaaS and SaaS. Give one advantage and one limitation of each model.
2. Describe the five cloud deployment models. Recommend a model for (a) a start-up building a mobile game, (b) a national bank, and (c) a group of state hospitals sharing patient records. Justify each choice.
3. Trace the history of virtualization from IBM CP-40 to Firecracker microVMs and explain why hardware-assisted virtualization was important for x86 servers.
4. Compare virtual machines and containers. Explain where Kubernetes and microservices fit into the picture.
5. Explain how Regions, Availability Zones, edge locations and regional edge caches work together to deliver a highly available, low-latency web application. Draw a Multi-AZ architecture.
6. Explain each of the 7 Rs of migration with a suitable example, and describe how an architect decides which R to apply to an application.
7. Describe the five steps of a cloud TCO calculation. Explain the difference between direct and indirect costs with three examples of each.
8. With a neat diagram, trace an HTTP request from a browser to a database and back, explaining what each tier does and how trace context is propagated.

D. Numerical and Case-Based Problems

1. A purchased server costs US$ 250,000 and US$ 0.60 per hour to power and maintain. The equivalent cloud instance costs US$ 30.60 per hour on demand. (a) Find the breakeven point in hours and in months (730 hours per month). (b) Recompute the breakeven if the workload runs only 12 hours a day and the cloud instance is stopped otherwise.
2. A company spends US$ 1.2 million per year on its data centre. Cloud estimates are US$ 0.9 million per year plus a one-time migration cost of US$ 0.6 million. Compute the cumulative cost of each option for years 1 to 4, and state in which year the cloud option becomes cheaper.
3. An e-commerce team on Amazon EC2 loses a day of orders after an unpatched operating system is compromised. Identify the responsibility that was misattributed, list the layers that belonged to the customer, and propose four preventive controls.

4. A trace shows: gateway 120 ms; Order service 180 ms; Payment and Inventory services called in parallel taking 900 ms and 350 ms; database 250 ms after both complete. Calculate the end-to-end latency and identify the critical path.

Lab Exercises

TRY IT YOURSELF: LAB 1: EXPLORE REGIONS AND AVAILABILITY ZONES

Using the AWS CLI (or AWS CloudShell), run the commands in Listing 1.8. Record the AZ names and AZ IDs for `ap-south-1` and `ap-south-2`. Explain why the AZ ID is more reliable than the AZ name when two accounts coordinate.

TRY IT YOURSELF: LAB 2: HYPERVISOR AND CONTAINERS ON YOUR LAPTOP

Install Oracle VirtualBox (a Type 2 hypervisor) and create an Ubuntu VM. Then install Docker inside the VM and run the container in Listing 1.6. Measure and compare the start-up time and disk usage of the VM and the container.

TRY IT YOURSELF: LAB 3: BUILD A TCO SPREADSHEET

Re-create the worked example of Section 1.8.5 in a spreadsheet with inputs for C, c, r and utilization. Plot cumulative cost against months and verify the breakeven values of 11.7, 14.9 and 21.5 months. Then add rows for staff, facilities and storage, and observe how the breakeven moves.

TRY IT YOURSELF: LAB 4: TRACE A REQUEST

Run two small services (for example in Python with Flask) instrumented with OpenTelemetry and export spans to a local Jaeger container. Call one service from the other, open the Jaeger UI and identify the trace ID, span IDs and the parent-child relationship.

Answers to Multiple-Choice Questions

Table 1.48 Answer key for Section A

Q	Ans	Q	Ans	Q	Ans
1	(c)	6	(c)	11	(b)
2	(b)	7	(b)	12	(a)
3	(b)	8	(b)	13	(c)
4	(c)	9	(c)	14	(b)

Q	Ans	Q	Ans	Q	Ans
5	(a)	10	(d)	15	(c)

Answers to Section D (outline). (1a) x = *250,000 ÷ (30.60 − 0.60) ≈ 8,333 hours ≈ 11.4 months. (1b) Effective cloud cost = 30.60 × 0.5 = 15.30 per calendar hour, so x* = 250,000 ÷ (15.30 − 0.60) ≈ 17,007 hours ≈ 23.3 months. (2) Cumulative on-premises: 1.2, 2.4, 3.6, 4.8 million; cloud: 1.5, 2.4, 3.3, 4.2 million; the options are equal at the end of year 2 and cloud is cheaper from year 3. (3) Guest OS patching and data backup were assumed to be the provider's; the customer owned the OS, data, IAM and network configuration. (4) 120 + 180 + max(900, 350) + 250 = 1,450 ms; the critical path is gateway, Order service, Payment service, database.

References and Further Reading

1. Mell, P. and Grance, T. *The NIST Definition of Cloud Computing*, NIST Special Publication 800-145. National Institute of Standards and Technology, 2011.
2. Popek, G. J. and Goldberg, R. P. “Formal Requirements for Virtualizable Third Generation Architectures.” *Communications of the ACM*, 17(7), 1974, pp. 412-421.
3. Amazon Web Services. *AWS Well-Architected Framework* (whitepaper and pillar papers). docs.aws.amazon.com/wellarchitected.
4. Amazon Web Services. *AWS Global Infrastructure*. aws.amazon.com/about-aws/global-infrastructure.
5. Amazon Web Services. *AWS Prescriptive Guidance: Migration strategies (the 7 Rs)* and *AWS Migration Lens*.
6. Amazon Web Services. *Shared Responsibility Model*. aws.amazon.com/compliance/shared-responsibility-model.
7. Microsoft. *Azure Well-Architected Framework* and *Shared responsibility in the cloud*. learn.microsoft.com/azure.
8. Google Cloud. *Google Cloud Architecture Framework* and *Shared responsibilities and shared fate*. cloud.google.com/architecture.
9. W3C. *Trace Context*, W3C Recommendation. www.w3.org/TR/trace-context.
10. OpenTelemetry Project. *OpenTelemetry Documentation*. opentelemetry.io/docs.
11. Kubernetes Project. *Kubernetes Documentation*. kubernetes.io/docs.
12. FinOps Foundation. *FinOps Framework*. finops.org/framework.